

## **Aula 03**

*DataPrev (Perfil 3: Desenvolvimento de Software) Desenvolvimento de Software - 2026 (Pós-Edital)*

Autor:  
**Vinicius Borges**

09 de Julho de 2026

# Índice

|                                                            |    |
|------------------------------------------------------------|----|
| 1) Java EE - Conceitos Básicos - Teoria .....              | 3  |
| 2) Java EE - Common Annotations - Teoria .....             | 11 |
| 3) Java EE - Conceitos Básicos - Questões Comentadas ..... | 14 |
| 4) Java EE - Conceitos Básicos - Lista de Questões .....   | 23 |
| 5) Java EE - JSF - Teoria .....                            | 28 |
| 6) Java EE - JSF - Questões Comentadas .....               | 37 |
| 7) Java EE - JSF - Lista de Questões .....                 | 53 |
| 8) Java EE - Primefaces .....                              | 61 |

## JAVA EE

Bem, galera... nosso foco aqui é Java EE<sup>1</sup>! Empresas de tecnologia da informação sofrem atualmente com a **altíssima competitividade**. Não é raro ver uma gigante, que todo mundo achava que seria eterna, desmoronando-se por conta de uma nova tecnologia que surgiu ou paradigma que apareceu! Ou alguém aí ainda usa IRC, ICQ, MSN para se comunicar?

É verdade, professor! As empresas de Tecnologia da Informação têm vida curta! Não são só elas! Hoje em dia, empresas de quaisquer áreas precisam de aplicações para satisfazer as suas **necessidades de negócio**, que estão se tornando cada vez mais complexas. E tudo isso se torna mais complicado com a globalização – as empresas estão cada vez mais espalhadas por cidades, países e continentes.

E, ainda assim, realizam seus negócios 24/7 por meio da internet, com um bocado de data centers e sistemas internacionalizados para lidar com diferentes línguas, moedas, fusos-horários, etc. E elas param de trabalhar em algum momento? Não! Estão sempre tentando **diminuir** seus custos, **tempo** de resposta de seus serviços, **armazenar** mais dados de maneira confiável e segura, entre outros.

E tudo isso de forma transparente para o cliente, que simplesmente acessa uma interface gráfica amigável achando que isso tudo é muito simples (mal sabem o que ocorre por trás). Pessoal, tudo tem que funcionar para o usuário não reclamar ou trocar de prestadora de serviço – e, claro, sem perder dinheiro, i.e., tem que haver prevenção de falhas, alta disponibilidade, redundância, escalabilidade e segurança.

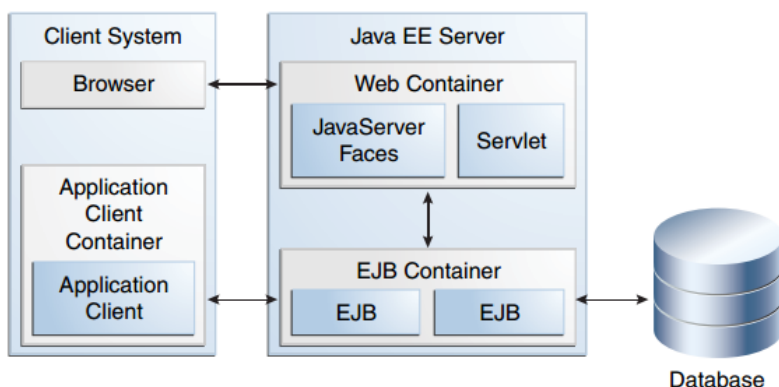
Além disso, as corporações têm de enfrentar constantes mudanças de requisitos, tecnologias, políticas, leis, etc. Em 2014, grande parte do que citamos é oferecido pelo **Java Enterprise Edition (Java EE)**. Mas, então, o que é de fato o Java EE? É um conjunto de especificações destinadas ao desenvolvimento de aplicações distribuídas, robustas, potentes, escaláveis, multicamadas e de alta disponibilidade.

Rapaziada, vamos ver agora algumas novidades trazidas pela **Plataforma Java EE 6**: conceito de profiles ou perfis; Java API for RESTful Web Services (JAX-RS); Managed Beans; Contexts and Dependency Injection (CDI); Dependency Injection for Java; Bean Validation; entre outras tecnologias concernentes a Enterprise JavaBeans, JavaServer Faces e Servlets.

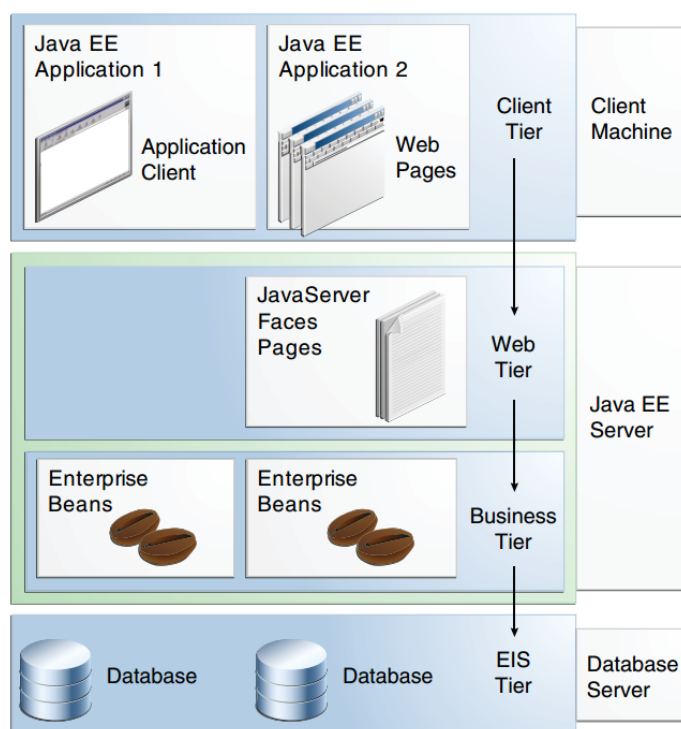
Como ele oferece tudo isso, nós veremos por meio do estudo de um assunto muito importante: Arquitetura Java EE – apresentada na imagem abaixo:

---

<sup>1</sup> Esse nome já mudou repetidas vezes! Inicialmente, chamava-se J2EE; depois foi modificado para JEE; e atualmente é conhecido como Java EE.



O Client System é a Camada do Cliente; Web Container é a Camada Web; o EJB Container é a Camada de Negócio; e o Database é a Camada de Dados<sup>2</sup>. No entanto, há quem condense a Camada Web e a Camada de Negócio em uma camada chamada **Servidor Java EE**, representada pelo retângulo maior à direita – veremos com detalhes mais à frente!



Vamos falar um pouco agora sobre o **Modelo de Aplicações Java EE**! Galera, Java EE é projetado para suportar aplicações que implementam serviços corporativos para clientes, empregados, fornecedores, parceiros e outros que demandem ou contribuem com a organização! Essas

<sup>2</sup> É também conhecida como Camada EIS (Enterprise Information System), que disponibiliza informações relevantes ao negócio e, diferente do que apresenta a imagem, não trata apenas do banco de dados, mas também de sistemas legados, processamento de transações de mainframe, sistemas externos, entre outros.

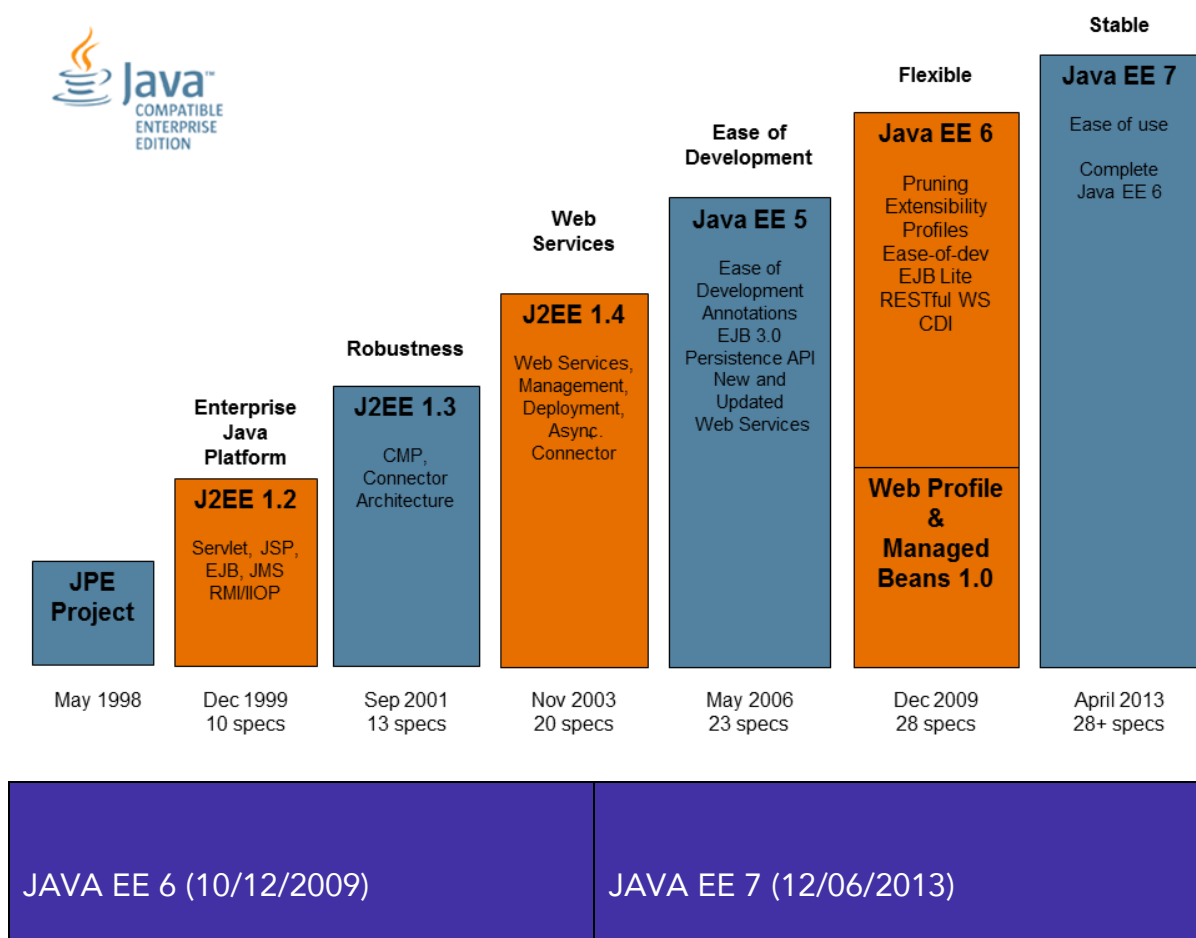
aplicações são inerentemente complexas, acessando dados de diversas fontes e distribuindo as aplicações entre os clientes.

O Modelo de Aplicações Java EE define uma arquitetura para implementação de serviços como aplicações multicamadas que fornecem escalabilidade, acessibilidade e gerenciabilidade necessários para aplicações corporativas. Dessa forma, a lógica de apresentação e a lógica de negócio são implementadas pelo desenvolvedor e os outros serviços são fornecidos pela plataforma **Java EE**!

Conforme mostra a imagem acima, existem duas aplicações multicamadas Java EE divididas em níveis descritos como se segue:

- Camada do Cliente: componentes rodam na Máquina Cliente;
- |   |                                                                                                                                                                             |   |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| { | <ul style="list-style-type: none"> <li>▪ Camada Web: componentes rodam no Servidor Java EE;</li> <li>▪ Camada de Negócio: componentes rodam no Servidor Java EE;</li> </ul> | } |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
- Camada EIS: software roda no Servidor EIS.

Galera, a imagem abaixo apresenta um pouco da evolução do Java EE e a tabela que segue apresenta as APIs do Java EE:



|                                                                            |                                                                            |
|----------------------------------------------------------------------------|----------------------------------------------------------------------------|
| -                                                                          | Java API for WebSocket                                                     |
| -                                                                          | Java API for JSON Processing                                               |
| Java Servlet 3.0                                                           | Java Servlet 3.1                                                           |
| JavaServer Faces (JSF) 2.0                                                 | JavaServer Faces (JSF) 2.2                                                 |
| Expression Language (EL) 2.2                                               | Expression Language (EL) 3.0                                               |
| JavaServer Pages (JSP) 2.2                                                 | JavaServer Pages (JSP) 2.3                                                 |
| JavaServer Pages Standard Tag Library (JSTL) 1.2                           | JavaServer Pages Standard Tag Library (JSTL) 1.2                           |
| -                                                                          | Batch Applications for the Java Platform                                   |
| -                                                                          | Concurrency Utilities for Java EE 1.0                                      |
| Contexts and Dependency Injection for Java 1.0                             | Contexts and Dependency Injection for Java 1.1                             |
| Dependency Injection for Java 1.0                                          | Dependency Injection for Java 1.0                                          |
| Bean Validation 1.0                                                        | Bean Validation 1.1                                                        |
| Enterprise JavaBeans (EJB) 3.1                                             | Enterprise JavaBeans (EJB) 3.2                                             |
| Interceptors 1.1                                                           | Interceptors 1.2                                                           |
| Java EE Connector Architecture 1.6                                         | Java EE Connector Architecture 1.7                                         |
| Java Persistence API (JPA) 2.0                                             | Java Persistence API (JPA) 2.1                                             |
| Common Annotations for the Java Platform 1.1                               | Common Annotations for the Java Platform 1.2                               |
| Java Message Service API (JMS) 1.1                                         | Java Message Service API (JMS) 2.0                                         |
| Java Transaction API (JTA) 1.1                                             | Java Transaction API (JTA) 1.2                                             |
| JavaMail API 1.4                                                           | JavaMail API 1.5                                                           |
| Java API for RESTful Web Services (JAX-RS) 1.1                             | Java API for RESTful Web Services (JAX-RS) 2.0                             |
| -                                                                          | Implementing Enterprise Web Services 1.3                                   |
| Java API for XML-Based Web Services (JAX-WS) 2.2                           | Java API for XML-Based Web Services (JAX-WS) 2.2                           |
| Web Services Metadata for the Java Platform 2.1                            | Web Services Metadata for the Java Platform                                |
| Java API for XML-based RPC (JAX-RPC) 1.1                                   | Java API for XML-based RPC (JAX-RPC) (Opcional) 1.1                        |
| Java APIs for XML Messaging (JAXM) 1.3                                     | Java APIs for XML Messaging 1.3                                            |
| Java API for XML Registries (JAXR) 1.0                                     | Java API for XML Registries (JAXR) 1.0                                     |
| Java Authentication Service Provider Interface for Containers (JASPIC) 1.0 | Java Authentication Service Provider Interface for Containers (JASPIC) 1.1 |
| Java Authorization Service Provider Contract for Containers (JACC) 1.4     | Java Authorization Service Provider Contract for Containers (JACC) 1.5     |

|                                              |                                               |
|----------------------------------------------|-----------------------------------------------|
| Java EE Application Deployment 1.2           | Java EE Application Deployment (Opcional) 1.2 |
| J2EE Management 1.1                          | J2EE Management 1.1                           |
| -                                            | Debugging Support for Other Languages 1.0     |
| Java Architecture for XML Binding (JAXB) 2.2 | Java Architecture for XML Binding (JAXB) 2.2  |
| -                                            | Java API for XML Processing (JAXP) 1.3        |
| -                                            | Java Database Connectivity 4.0                |
| -                                            | Java Management Extensions (JMX) 2.0          |
| -                                            | JavaBeans Activation Framework (JAF) 1.1      |
| -                                            | Streaming API for XML (StAX) 1.0              |
| Managed Beans 1.0                            | -                                             |
| Web Services 1.3                             | -                                             |
| Debuggin Support for Other Languages 1.0     | -                                             |

A versão Java EE 6 traz o conceito de profile (ou perfil)! O que é isso, professor? Um perfil busca definir um subconjunto das tecnologias dentre aquelas da plataforma Java EE. Como assim? Bem, pensem comigo: cada aplicação tem sua particularidade, portanto não é necessário implementar obrigatoriamente todas as tecnologias da plataforma, i.e., eu posso criar perfis – cada um com sua configuração!

Imaginem que vamos fazer um sisteminha pequeno! Eu preciso implementar tudo que está na plataforma? Não, posso criar um perfil que implementa somente um subconjunto de funcionalidades! Existem dois perfis importantes: Web Profile e Full Profile! O primeiro perfil é um subconjunto do segundo e ajuda desenvolvedores a criarem aplicações mais leves que podem rodar em um Servlet Container.

| Plataforma Java EE                               | WEB | FULL |
|--------------------------------------------------|-----|------|
| Java Servlet                                     | ✓   | ✓    |
| Java Server Faces (JSF)                          | ✓   | ✓    |
| Java Server Pages (JSP)                          | ✓   | ✓    |
| Expression Language (EL)                         | ✓   | ✓    |
| Standard Tag Library for JavaServer Pages (JSTL) | ✓   | ✓    |
| Debugging Support for Other Languages            | ✓   | ✓    |

|                                                            |   |   |
|------------------------------------------------------------|---|---|
| Contexts and Dependency Injection for the Java EE Platform | ✓ | ✓ |
| Dependency Injection for Java                              | ✓ | ✓ |
| Enterprise JavaBeans (EJB)                                 | ✓ | ✓ |
| Java Persistence API                                       | ✓ | ✓ |
| Common Annotations for the Java Platform                   | ✓ | ✓ |
| Java Transaction API                                       | ✓ | ✓ |
| Bean Validation                                            | ✓ | ✓ |
| Java EE Connector Architecture                             |   | ✓ |
| Java API for RESTful Web Services (JAX-RS)                 |   | ✓ |

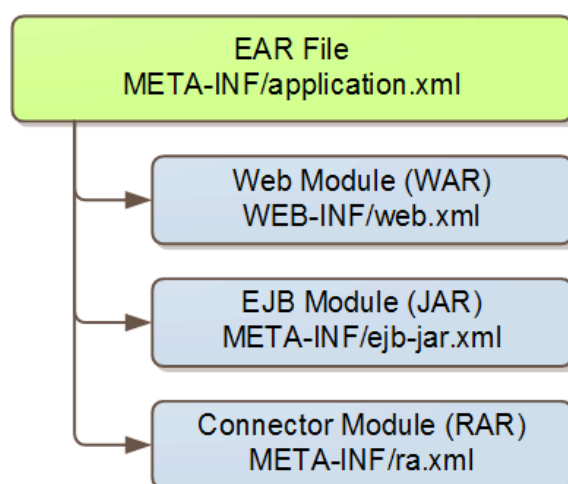
Observem que a tabela abaixo apresenta o EJB 3.1 como parte do Web Profile. Na verdade, no Web Profile, trata-se do EJB 3.1 Lite, que é **mais leve**. Como assim, professor? Assim como os perfis, ele possui um subconjunto dos features do EJB 3.1 Full. Por que? Porque é uma API utilizada especificamente para aplicações web. Vejam a diferença de acordo com a tabela abaixo.

Percebam que o EJB 3.1 Lite deixa de fora funcionalidades que são pouco utilizadas em aplicações web. De forma similar o Web Profile não oferece suporte a JAX-WS, JAX-RPC, JAXR, SAAJ, JAX-RS, JAXB, JMS, JAAS, JASPIC, JACC, JCA, JavaMail, Management Specification e Deployment Specification – além disso, ele não oferece suporte a Arquivos EAR (apenas Arquivos WAR).

| Feature                   | EJB Lite | EJB |
|---------------------------|----------|-----|
| Stateless beans           | ✓        | ✓   |
| Stateful beans            | ✓        | ✓   |
| Singleton beans           | ✓        | ✓   |
| Message driven beans      |          | ✓   |
| Remoting                  |          | ✓   |
| Web service (SOAP/REST)   |          | ✓   |
| Asynchronous invocation   |          | ✓   |
| Interceptors              | ✓        | ✓   |
| Declarative security      | ✓        | ✓   |
| Declarative transactions  | ✓        | ✓   |
| Programmatic transactions | ✓        | ✓   |
| Timer service             |          | ✓   |
| EJB 2 support             |          | ✓   |
| CORBA interoperability    |          | ✓   |

Table 2: EJB and EJB Lite Feature Comparison

Para finalizar, vamos entender algumas coisinhas! O processo de implantar (para alguns, instalar) uma aplicação em um Servidor Java EE é chamado Deploy ou Deployment. Sabe-se que componentes são agrupados em módulos, compactados em .ZIP e, na Implantação, mapeia-se cada componente do Java EE para seu contêiner correspondente. Existem três tipos básicos de módulo:



| MÓDULO | DESCRIÇÃO                                                                                                                                                         |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| EAR    | Também chamado Enterprise Application Archives, contém a aplicação completa, com todos os seus módulos e componentes. É composta por vários arquivos .war e .jar. |
| WAR    | Também chamado Web Application Archives, contém a Aplicação Web (JSP, HTML, Servlets, Arquivos de Configuração, Imagens, etc) – é o que forma uma página em si.   |
| JAR    | Também chamado Java Application Archives, contém a Aplicação EJB, Aplicação Cliente e Applets <sup>3</sup> , além de arquivos de configuração dos aplicativos.    |
| RAR    | Também chamado Resource Adapter, contém interfaces, classes, bibliotecas, etc.                                                                                    |

---

<sup>3</sup> A bem da verdade, todos os módulos são Arquivos JAR com a extensão modificada. *Por que essa mudança?* Para que o servidor possa diferenciar o que está sendo implantado.

## COMMON ANNOTATIONS

Galera, o objeto primário dessa especificação é **definir** um pequeno conjunto de **anotações** disponíveis para utilização dentro de **outras especificações**. Dessa forma, evitam-se redundâncias ou duplicações desnecessárias entre anotações definidas em especificações diferentes, incluindo até mesmo aquelas utilizadas em plataformas distintas. *Bacana?* Vamos ver as anotações:

- `javax.annotation.Generated`:

Essa anotação é utilizada para marcar o código-fonte que foi gerado. Ela pode ser especificada em classes, métodos ou campos. Também pode ser usada para diferenciar o código escrito pelo programador e código gerado automaticamente em um mesmo arquivo. Possui três elementos: `value`, que é o nome do gerador de código; `date`, que é a data de geração; e `comments`, que são simplesmente comentários.

- `javax.annotation.Resource`:

Essa anotação é utilizada para declarar uma referência a um recurso. Ela pode ser especificada em classes, métodos ou campos. Quando aplicada a métodos ou campos, o contêiner injetará uma instância do recurso requisitado na aplicação quando ela for inicializada. Se for aplicada a classes, ela declara um recurso que a aplicação procurará em tempo de execução.

- `javax.annotation.Resources`:

Essa anotação é utilizada para declarar uma referência a um recurso. Ela age como um contêiner para múltiplas declarações de recursos. Pessoal, não confundam com a anterior! Uma é `Resource` e a outra é `Resources`. Quando se têm mais de uma declaração de referência a um recurso, utilizamos a segunda! Não há muito segredo nessa anotação.

- `javax.annotation.PostConstruct`:

Essa anotação é utilizada em um método que necessita ser executado após uma injeção de dependência terminar para executar uma inicialização. Ele é invocado antes de a classe ser colocada em serviço. Essa anotação deve ser suportada em todas as classes que suportam injeção de dependência. O método em que essa anotação é aplicada deve satisfazer diversos critérios.

- `javax.annotation.PreDestroy`:

Essa anotação é utilizada em métodos como uma notificação de retorno para sinalizar que uma instância está no processo de ser removida pelo contêiner. O método com essa anotação é tipicamente utilizado para liberar recursos que esteja mantendo. O método em que essa anotação é aplicada deve satisfazer diversos critérios.

- `javax.annotation.Priority`:

Essa anotação pode ser aplicada a classes para indicar em que ordem as classes devem ser utilizadas. O efeito do uso dessa anotação em uma instância particular é definido por outras especificações que definem o uso de uma classe específica – cada classe específica vai dizer qual deverá ser a prioridade de chamada das outras classes. *Entenderam?*

- `javax.annotation.security.RunAs`:

Essa anotação define a função da aplicação durante a execução em um Contêiner Java EE. Ela pode ser especificada em uma classe. Isso permite que os desenvolvedores executem uma aplicação sob uma função particular. A função deve ser mapeada para as informações de usuário/grupo no domínio de segurança do contêiner. O elemento `value`, nesse elemento, é uma função de segurança.

- `javax.annotation.security.RolesAllowed`:

Essa anotação especifica as funções de segurança permitidas para acessar métodos em uma aplicação. O elemento `value` dessa anotação é uma lista de nomes de funções de segurança. Essa anotação pode ser especificada em uma classe, quando se aplica a todos os métodos dessa classe; ou em um método, quando se aplica apenas ao método específico.

- `javax.annotation.security.PermitAll`:

Essa anotação especifica que todas as funções de segurança estão autorizadas a invocar métodos específicos. Em outras palavras, podemos dizer que os métodos específicos estão “unchecked”. Essa anotação pode ser especificada em uma classe, quando se aplica a todos os métodos dessa classe; ou em um método, quando se aplica apenas ao método específico.

- `javax.annotation.security.DenyAll`:

Essa anotação especifica que nenhuma função de segurança está autorizada a invocar métodos específicos. Em outras palavras, podemos dizer que os métodos serão excluídos no Contêiner Java EE. Essas três últimas anotações definem quais funções de segurança estão autorizadas a acessar os métodos em que elas foram aplicadas.

- `javax.annotation.security.DeclareRoles`:

Essa anotação é utilizada para especificar as funções de segurança de uma aplicação. Ela pode ser especificada em uma classe. Ela tipicamente é utilizada para definir funções que poderiam ser testadas de dentro de métodos das classes anotadas. Ela também pode ser utilizada para declarar papéis que não estão implicitamente declarados.

- `javax.annotation.sql.DataSourceDefinition`:

Essa anotação é utilizada para definir um contêiner DataSource e para ser registrada com JNDI (Java Naming and Directory Interface). O DataSource pode ser configurado através da definição dos elementos de anotação para propriedades de DataSource comumente utilizadas. Ele é registrado sob o nome especificado no elemento name e determina a acessibilidade da fonte de dados de outros componentes.

- `javax.annotation.sql.DataSourceDefinitions`:

O mesmo que o anterior, porém é possível definir vários DataSource.

- `javax.annotation.ManagedBean`:

Essa anotação é utilizada para declarar um Managed Bean. *O que é um Managed Bean?* É um contêiner de objetos gerenciados que suportam um pequeno conjunto de serviços básicos, tais como injeção de recursos, chamadas de ciclo de vida e interceptadores. Um Managed Bean pode opcionalmente possuir um nome, isto é, uma string especificada no elemento value.

### OBSERVAÇÕES

Prezados, sabe quantos editais eu encontrei cobrando esse assunto? Somente um! Sabe quantas questões eu encontrei sobre esse assunto? Absolutamente nenhuma! Logo, saibam dosar seus níveis de atenção e estudos em cada disciplina.

## QUESTÕES COMENTADAS – JAVA EE - MULTIBANCAS

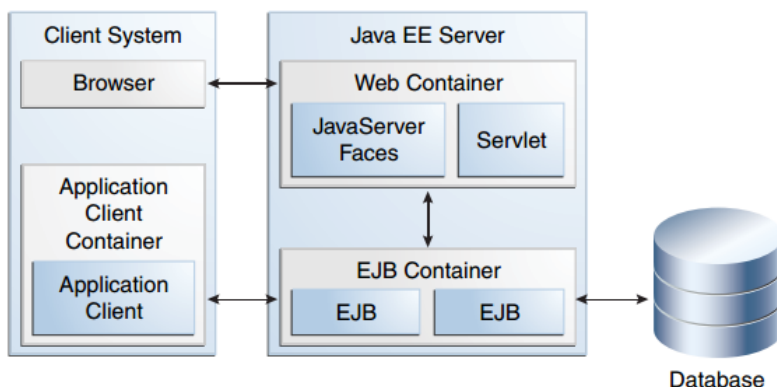
1. (CESPE – 2009 – INMETRO – Analista de Sistemas) São exemplos de tipos de componentes de software reusáveis desenvolvidos na plataforma JEE: JSP (Java Server Page); biblioteca de tags; Servlet; EJB. O grau de reúso provido por esses componentes, EJBs e JSPs, é usualmente superior a bibliotecas de TAG.

### Comentários:

Galera, vamos responder isso intuitivamente! Como uma Página JSP ou um Componente EJB poderia oferecer maior reusabilidade que uma biblioteca? Ora, essa é uma das principais características de uma biblioteca: sua reusabilidade! Logo, isso não faz sentido! Bibliotecas de Tags são mais reusáveis. **Gabarito: E**

2. (CESPE – 2005 – SERPRO – Analista de Sistemas) A tecnologia Enterprise JavaBeans (EJB) é uma arquitetura de componentes do tipo cliente que atua na plataforma J2EE.

### Comentários:



|                                |                                |
|--------------------------------|--------------------------------|
| JAVA EE 6 (10/12/2009)         | JAVA EE 7 (12/06/2013)         |
| Enterprise JavaBeans (EJB) 3.1 | Enterprise JavaBeans (EJB) 3.2 |

Conforme vimos em aula, Enterprise Java Bean (EJB) não é uma arquitetura, é um componente da Arquitetura J2EE. Além disso, é do tipo Servidor (veja a imagem acima). **Gabarito: E**

3. (CESPE - 2010 – TCU – Auditor Federal de Controle Externo) A web profile da plataforma JEE apresenta, em relação ao perfil application server definido em edições anteriores da plataforma Java, as seguintes vantagens: fornece suporte para POJOs (Plain Old Java Objects) e Annotations; possui modelo de empacotamento de componentes mais simples; a configuração dos seus descritores XML (extensible markup language) é mais fácil; é aderente ao padrão SOA.

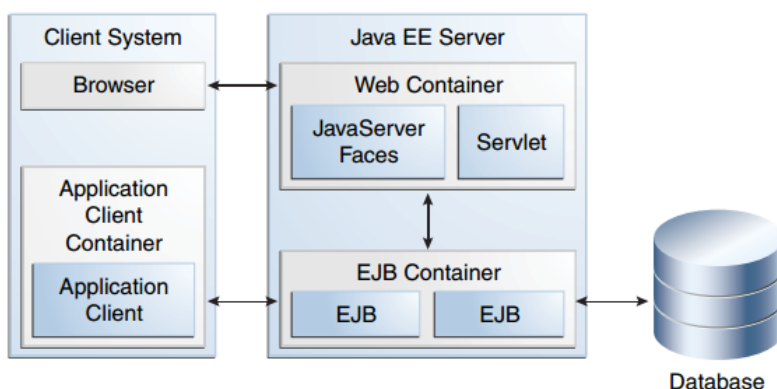
#### Comentários:

Imaginem que vamos fazer um sisteminha pequeno! Eu preciso implementar tudo que está na plataforma? Não, posso criar um perfil que implementa somente um subconjunto de funcionalidades! Existem dois perfis importantes: Web Profile e Full Profile! O primeiro perfil é um subconjunto do segundo e ajuda desenvolvedores a criarem aplicações mais leves que podem rodar em um Servlet Container.

Conforme vimos em aula, primeiro, não existe Application Server Profile – existe apenas Web Profile e Full Profile. Segundo, o conceito de Perfis foi introduzido apenas no Java EE 6 – eu calculo que ele esteja considerando Full Profile como Application Server Profile. Terceiro, POJOs e Annotations são tecnologias do Java EE 5. Quarto, pode-se dizer que é aderente ao SOA por conta do JAX-RS, no entanto o Full Profile também é (inclusive é aderente ao JAX-RS também). Logo, a questão está errada desde o início. **Gabarito: E**

4. (CESPE - 2010 – TRE/MT – Analista Judiciário – Tecnologia da Informação – A) Clientes J2EE são necessariamente páginas web dinâmicas que normalmente não fazem acessos a banco de dados, nem executam regras de negócio complexas.

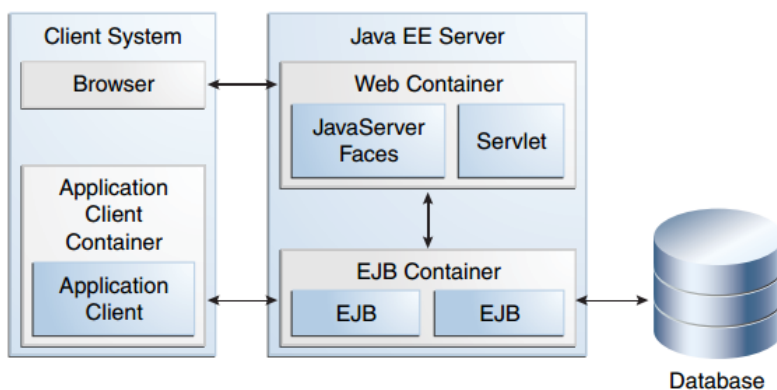
#### Comentários:



Conforme vimos em aula, os clientes Java EE não são necessariamente Páginas Web Dinâmicas (Browser). A imagem acima mostra que eles podem ser também uma Aplicação Cliente. **Gabarito: E**

5. (CESPE - 2010 – TRE/MT – Analista Judiciário – Tecnologia da Informação – D) Um componente J2EE é uma unidade funcional de software autocontida, escrito na linguagem de programação Java e executado exclusivamente em servidores.

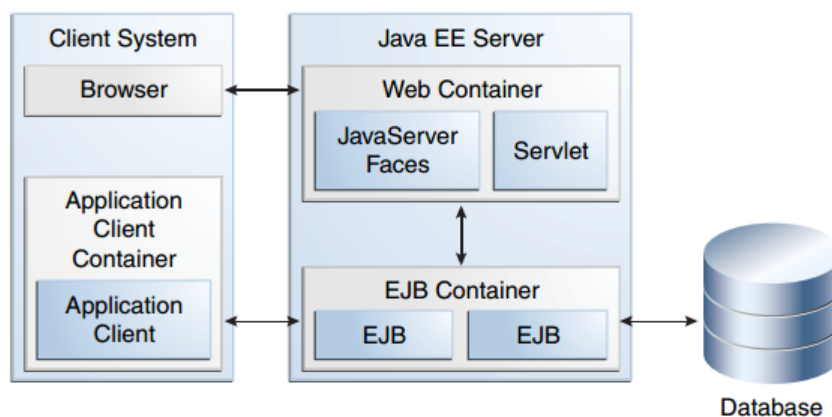
Comentários:



Conforme vimos em aula, um componente Java EE é uma unidade autocontida, porque pode ser reusada sem a necessidade de incluir ou depender de outros componentes. Ademais, eles são escritos na linguagem de programação Java, no entanto não são necessariamente executados exclusivamente em servidores, podem ser executados no cliente (conforme imagem acima). **Gabarito: E**

6. (CESPE - 2011 – PREVIC – Analista de Sistemas) Em uma aplicação multicamadas na plataforma Java EE, servlets, JavaServer Faces e JSP consistem em tecnologias utilizadas na camada web.

Comentários:



Conforme vimos em aula, a Camada Web é composta por JSP, JSF e Servlets. **Gabarito: C**

7. (ESAF – 2012 – CGU – Analista de Finanças e Controle) Os níveis da plataforma J2EE são:

- a) Patrocinador. Web. Negócios. Sistemas de Computação Corporativos.
- b) Cliente. Web. Negócios. Sistemas de Informação Corporativos.
- c) Cliente. Interno. Externo. Negócios.
- d) Fornecedor. Web. Político. Sistemas de Informação Camada.
- e) Cliente. Stakeholders. Negócios. Background corporativo.

#### Comentários:

Camada do Cliente: componentes rodam na Máquina Cliente;

Camada Web: componentes rodam no Servidor Java EE;

Camada de Negócio: componentes rodam no Servidor Java EE;

Camada EIS: software roda no Servidor EIS.

Conforme vimos em aula, os níveis são: Cliente, Web, Negócios e Sistemas de Informação Corporativos (EIS). **Gabarito: B**

8. (CESGRANRIO – 2008 – BNDES – Analista de Sistemas) Uma aplicação empresarial contendo componentes EJB e módulos web deverá ser publicada em um servidor de aplicações compatível com J2EE. No contexto do empacotamento dessa aplicação para publicação (deploy), é correto afirmar que:

- a) não há como juntar componentes EJB e módulos web em uma mesma aplicação, pois deverão ser publicados separadamente.

- b) um arquivo EAR poderá conter arquivos WAR e JAR representativos dos módulos web e EJB.
- c) o tamanho do pacote, em bytes, sempre fica maior que o código original, em virtude do algoritmo empregado no empacotamento da aplicação em um arquivo EAR.
- d) módulos web não devem ser empacotados, pois isso inviabiliza seu acesso pela Internet.
- e) arquivos JAR servem apenas para empacotar componentes EJB.

### Comentários:

Para finalizar, vamos entender algumas coisinhas! O processo de implantar (para alguns, instalar) uma aplicação em um Servidor Java EE é chamado Deployment. Sabe-se que componentes são agrupados em módulos, compactados em .ZIP e, na Implantação, mapeia-se cada componente da Arquitetura Java EE para seu contêiner correspondente. Existem três tipos básicos de módulo:

Conforme vimos em aula, a primeira opção está errada, porque pode-se junta ambos em um Arquivo EAR; a segunda opção está correta e justifica a primeira; a terceira opção está errada, porque são arquivos compactados em .ZIP; a quarta opção está errada, porque simplesmente não faz nenhum sentido; e a última opção está errada porque arquivo JAR pode empacotar componentes EJB, Cliente e Applet. **Gabarito: B**

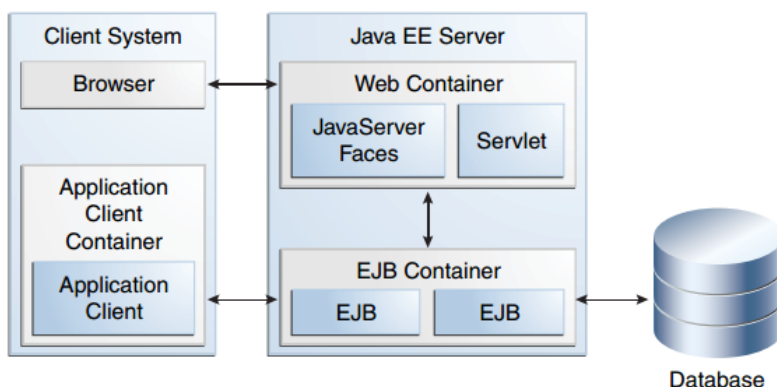
### 9. (FCC – 2011 – TRT/19 – Analista de Sistemas) A especificação Java EE define os seguintes componentes:

- I. Clientes da aplicação (Application Clients) e applets.
- II. Java Servlet, JavaServer Faces e JavaServer Pages.
- III. Enterprise Javabeans (EJB).

Os componentes I, II e III rodam, respectivamente, em:

- a) cliente, cliente, servidor.
- b) servidor, cliente, servidor.
- c) cliente, servidor, servidor.
- d) servidor, cliente, cliente.
- e) cliente, servidor, cliente.

### Comentários:



Conforme vimos em aula, tanto aplicações clientes como applets rodam no cliente; Servlets, JSF e JSP rodam no Servidor, assim como o EJBs. **Gabarito: C**

10.(FCC - 2011 - TRT - 1ª REGIÃO (RJ) - Analista Judiciário - Tecnologia da Informação) J2EE é uma plataforma de programação para servidores na linguagem de programação Java, que integra uma série de especificações e containers, cada uma com funcionalidades distintas. Nesse contexto, é correto afirmar que são integrantes do J2EE:

- a) Servlets, Jcompany e JSP.
- b) JDBC, JSP, EJBs.
- c) EJBs, Servlets e JBoss.
- d) JDBC, Hibernate e JPA.
- e) JSP, JSF e Eclipse.

**Comentários:**

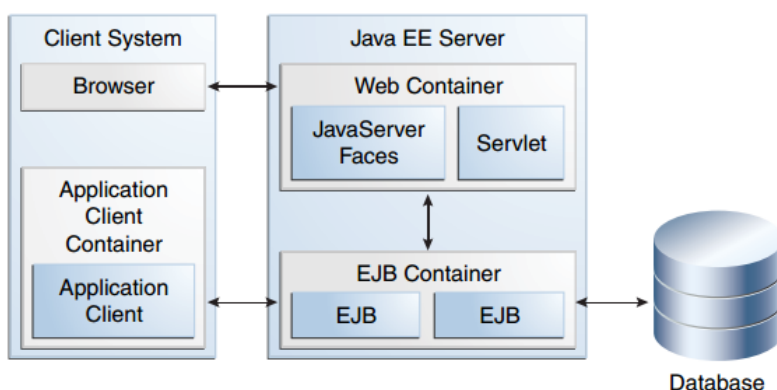
| JAVA EE 6 (10/12/2009)         | JAVA EE 7 (12/06/2013)         |
|--------------------------------|--------------------------------|
| JavaServer Pages (JSP) 2.2     | JavaServer Pages (JSP) 2.3     |
| Enterprise JavaBeans (EJB) 3.1 | Enterprise JavaBeans (EJB) 3.2 |
| -                              | Java Database Connectivity 4.0 |

Conforme vimos em aula, trata-se do JDBC, JSP e EJB! Muitas pessoas me perguntam: "Professor, Hibernate não é integrante do J2EE?". Galera, o Hibernate é um framework que implementa a especificação JPA. Logo, ele não faz parte do J2EE ou Java EE. Beleza? **Gabarito: B**

11.(FCC - 2010 - TRT - 8ª Região (PA e AP) - Analista Judiciário - Tecnologia da Informação) O Contêiner J2EE que fornece aos desenvolvedores o ambiente para rodar Java Server Pages (JSPs) e servlets é:

- a) Applet (Applet container).
- b) Enterprise Java Beans (EJB).
- c) Interface (Interface container).
- d) do cliente do aplicativo (Application client container).
- e) Web (Web container).

Comentários:

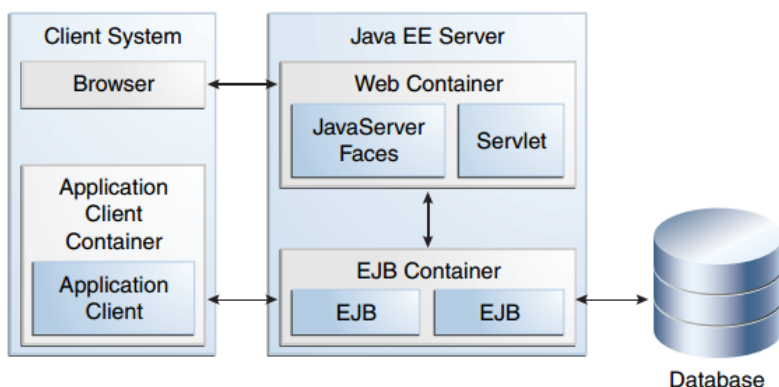


Conforme vimos em aula, o Contêiner Web é o responsável por rodar JSP/Servlet! **Gabarito: E**

12.(FCC - 2010 - TCE-SP - Agente da Fiscalização Financeira - Informática - Suporte de Web) São apenas tipos de componentes executados em servidores Web:

- a) Beans, Servlets e J2EE.
- b) JVM, Servlets e JSP.
- c) Beans, Servlets e JSP.
- d) Beans, Swing e JSP.
- e) Beans, Swing e JVM.

Comentários:

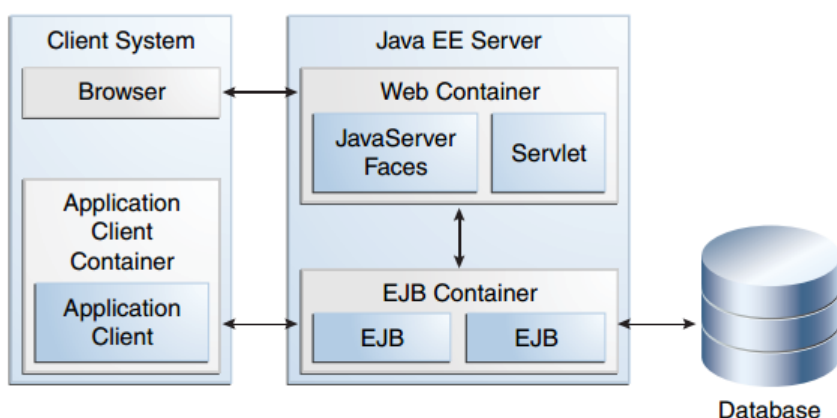


Conforme vimos em aula, JSP e Servlets são fáceis! E os beans? Pois é, excepcionalmente eles podem ser executados em Servidores Web (Contêiner Web). **Gabarito: C**

13. (FCC - 2014 – TRT/2 – Analista de Sistemas) Um contêiner Java EE pode oferecer serviços como gestão de memória, ciclo de vida e estado de objetos, conexões, transações, serviços de nomes, segurança, tolerância a falhas, integração, clustering, alta disponibilidade, confiabilidade e web services. Um servidor Java EE completo disponibiliza dois tipos principais de contêiner, que são:

- a) Contêiner MVC e Contêiner EJB.
- b) Applet Container e Web Container.
- c) Contêiner Web e Contêiner EJB.
- d) Servlet Container e JSP Container.
- e) Application Client Container e Web Container.

**Comentários:**



Conforme vimos em aula, disponibiliza o Contêiner Web e Contêiner EJB! **Gabarito: C**

14. (FCC – 2012 – TJ/PE – Analista de Sistemas) Sobre a plataforma Java EE 6, é correto afirmar:

- a) Simplifica a implantação sem a necessidade de descritores de implantação, com exceção do descritor de implantação exigido pela especificação servlet, o arquivo web.xml.
- b) Necessita do descritor de implantação ejb-jar.xml e entradas relacionadas aos web services no arquivo web.xml.
- c) Faz uso de anotações (annotations). Anotações são modificadores Java, semelhantes aos públicos e privados, que devem ser especificados nos arquivos de configuração XML.
- d) A especificação EJB 3, que é um subconjunto da especificação Java EE, define anotações apenas para o tipo bean.
- e) Anotações são marcados com um caracter # (cerquilha).

### Comentários:

- (a) Correto. Pessoal, a plataforma Java EE realmente simplifica a implantação removendo a necessidade de descritores de implantação, mas há uma exceção: o arquivo web.xml;
- (b) Descritores de implantação, como o ejb-jar.xml e entradas relacionadas aos Web services no web.xml, já estão obsoletos – não se usa mais!
- (c) A Plataforma Java EE utiliza Anotações, que são modificadores Java, semelhantes aos públicos e privados. No entanto, eles são especificados no código!
- (d) Ele define anotações para o tipo Bean, tipo de Interface, referências de recurso, atributos de transação, segurança, etc;
- (e) Essa ele entregou! Anotações são marcados com @. **Gabarito: A**

## LISTA DE QUESTÕES – JAVA EE - MULTIBANCAS

1. (CESPE – 2009 – INMETRO – Analista de Sistemas) São exemplos de tipos de componentes de software reusáveis desenvolvidos na plataforma JEE: JSP (Java Server Page); biblioteca de tags; Servlet; EJB. O grau de reuso provido por esses componentes, EJBs e JSPs, é usualmente superior a bibliotecas de TAG.
2. (CESPE – 2005 – SERPRO – Analista de Sistemas) A tecnologia Enterprise JavaBeans (EJB) é uma arquitetura de componentes do tipo cliente que atua na plataforma J2EE.
3. (CESPE - 2010 – TCU – Auditor Federal de Controle Externo) A web profile da plataforma JEE apresenta, em relação ao perfil application server definido em edições anteriores da plataforma Java, as seguintes vantagens: fornece suporte para POJOs (Plain Old Java Objects) e Annotations; possui modelo de empacotamento de componentes mais simples; a configuração dos seus descritores XML (extensible markup language) é mais fácil; é aderente ao padrão SOA.
4. (CESPE - 2010 – TRE/MT – Analista Judiciário – Tecnologia da Informação – A) Clientes J2EE são necessariamente páginas web dinâmicas que normalmente não fazem acessos a banco de dados, nem executam regras de negócio complexas.
5. (CESPE - 2010 – TRE/MT – Analista Judiciário – Tecnologia da Informação – D) Um componente J2EE é uma unidade funcional de software autocontida, escrito na linguagem de programação Java e executado exclusivamente em servidores.
6. (CESPE - 2011 – PREVIC – Analista de Sistemas) Em uma aplicação multicamadas na plataforma Java EE, servlets, JavaServer Faces e JSP consistem em tecnologias utilizadas na camada web.
7. (ESAF – 2012 – CGU – Analista de Finanças e Controle) Os níveis da plataforma J2EE são:
  - a) Patrocinador. Web. Negócios. Sistemas de Computação Corporativos.
  - b) Cliente. Web. Negócios. Sistemas de Informação Corporativos.
  - c) Cliente. Interno. Externo. Negócios.
  - d) Fornecedor. Web. Político. Sistemas de Informação Camada.
  - e) Cliente. Stakeholders. Negócios. Background corporativo.

8. (CESGRANRIO – 2008 – BNDES – Analista de Sistemas) Uma aplicação empresarial contendo componentes EJB e módulos web deverá ser publicada em um servidor de aplicações compatível com J2EE. No contexto do empacotamento dessa aplicação para publicação (deploy), é correto afirmar que:

- a) não há como juntar componentes EJB e módulos web em uma mesma aplicação, pois deverão ser publicados separadamente.
- b) um arquivo EAR poderá conter arquivos WAR e JAR representativos dos módulos web e EJB.
- c) o tamanho do pacote, em bytes, sempre fica maior que o código original, em virtude do algoritmo empregado no empacotamento da aplicação em um arquivo EAR.
- d) módulos web não devem ser empacotados, pois isso inviabiliza seu acesso pela Internet.
- e) arquivos JAR servem apenas para empacotar componentes EJB.

9. (FCC – 2011 – TRT/19 – Analista de Sistemas) A especificação Java EE define os seguintes componentes:

I. Clientes da aplicação (Application Clients) e applets.

II. Java Servlet, JavaServer Faces e JavaServer Pages.

III. Enterprise Javabeans (EJB).

Os componentes I, II e III rodam, respectivamente, em:

- a) cliente, cliente, servidor.
- b) servidor, cliente, servidor.
- c) cliente, servidor, servidor.
- d) servidor, cliente, cliente.
- e) cliente, servidor, cliente.

10. (FCC - 2011 - TRT - 1ª REGIÃO (RJ) - Analista Judiciário - Tecnologia da Informação) J2EE é uma plataforma de programação para servidores na linguagem de programação Java, que integra uma série de especificações e containers, cada uma com funcionalidades distintas. Nesse contexto, é correto afirmar que são integrantes do J2EE:

- a) Servlets, Jcompany e JSP.
- b) JDBC, JSP, EJBs.
- c) EJBs, Servlets e JBoss.
- d) JDBC, Hibernate e JPA.
- e) JSP, JSF e Eclipse.

**11.(FCC - 2010 - TRT - 8ª Região (PA e AP) - Analista Judiciário - Tecnologia da Informação) O Contêiner J2EE que fornece aos desenvolvedores o ambiente para rodar Java Server Pages (JSPs) e servlets é:**

- a) Applet (Applet container).
- b) Enterprise Java Beans (EJB).
- c) Interface (Interface container).
- d) do cliente do aplicativo (Application client container).
- e) Web (Web container).

**12.(FCC - 2010 - TCE-SP - Agente da Fiscalização Financeira - Informática - Suporte de Web) São apenas tipos de componentes executados em servidores Web:**

- a) Beans, Servlets e J2EE.
- b) JVM, Servlets e JSP.
- c) Beans, Servlets e JSP.
- d) Beans, Swing e JSP.
- e) Beans, Swing e JVM.

**13.(FCC - 2014 – TRT/2 – Analista de Sistemas) Um contêiner Java EE pode oferecer serviços como gestão de memória, ciclo de vida e estado de objetos, conexões, transações, serviços de nomes, segurança, tolerância a falhas, integração, clustering, alta disponibilidade, confiabilidade e web services. Um servidor Java EE completo disponibiliza dois tipos principais de contêiner, que são:**

- a) Contêiner MVC e Contêiner EJB.
- b) Applet Container e Web Container.
- c) Contêiner Web e Contêiner EJB.
- d) Servlet Container e JSP Container.
- e) Application Client Container e Web Container.

**14.(FCC – 2012 – TJ/PE – Analista de Sistemas) Sobre a plataforma Java EE 6, é correto afirmar:**

- a) Simplifica a implantação sem a necessidade de descritores de implantação, com exceção do descritor de implantação exigido pela especificação servlet, o arquivo web.xml.
- b) Necessita do descritor de implantação ejb-jar.xml e entradas relacionadas aos web services no arquivo web.xml.

- c) Faz uso de anotações (annotations). Anotações são modificadores Java, semelhantes aos públicos e privados, que devem ser especificados nos arquivos de configuração XML.
- d) A especificação EJB 3, que é um subconjunto da especificação Java EE, define anotações apenas para o tipo bean.
- e) Anotações são marcados com um caracter # (cerquilha).

## GABARITO

GABARITO



1. E  
2. E  
3. E  
4. E  
5. E

6. C  
7. B  
8. B  
9. C  
10. B

11. E  
12. C  
13. C  
14. A

## JAVA SERVER FACES (JSF)

Galera, vou contar uma historinha para vocês! Durante muito tempo, usuários se acostumaram com Aplicações Desktop. O que é isso, professor? É aquele programinha que você baixa e instala em seu computador local, acessando diretamente bancos de dados ou gerenciadores de arquivos. Para criar essas aplicações eram utilizadas as tecnologias Visual Basic, Delphi, Swing (Java), etc.

Pois é, esses programas em geral não necessitam de um navegador para rodar! Eles são construídos como um conjunto de componentes oferecidos pela plataforma de desenvolvimento para cada sistema operacional e estão associados a eventos, ações ou procedimentos que executam lógicas de negócio. Muitas das vezes, são componentes muito ricos.

No entanto, sabe-se que Aplicações Desktop sofrem com problemas de manutenção e gerenciabilidade. Vejam que as regras de negócio rodam no cliente – aliás, uma cópia integral da aplicação está no cliente! Logo, se deu pau em alguma funcionalidade, eu tenho que propagar as alterações para todas as máquinas que têm o programa, visto que as regras de negócio estão no cliente!

Para resolver esse tipo de problema, surgiram as Aplicações Web. Elas rodam em um servidor central onde os usuários podem acessá-las por meio de um navegador (Chrome, Firefox, etc) e um protocolo HTTP. Nesse caso, todas as regras de negócio da aplicação se encontram no Servidor, sendo muito mais fácil gerenciá-las e, eventualmente, depurá-las.

Ora, deu pau em alguma funcionalidade, eu vou lá no servidor central e conserto – não preciso ir em todas as máquinas que têm a aplicação! Claro, nem tudo são flores, é necessário conhecer diversas tecnologias, linguagens, scripts, entre outros – além de seguir um modelo de requisição/resposta. Agora vejam que interessante, eu posso combinar esses dois universos.

É possível unir as melhores características desses dois mundos, com componentes ricos, abstraindo protocolos, etc. Vocês já ouviram falar da Arquitetura Model-View-Controller (MVC)? Pois é, trata-se de uma arquitetura que divide os componentes de uma aplicação em camadas independentes (Modelo, Visão e Controle)! O Controle faz o meio campo entre a Visão e o Modelo.

Bem, ela foi amplamente utilizada em Aplicações Desktop até que um dia alguém pensou: Poxa, por que não utilizá-la na web? E resolveu assim fazê-lo! Como assim? Alguém decidiu então combinar a Arquitetura MVC com o modelo tradicional de desenvolvimento de páginas web dinâmicas! Qual seria esse modelo, professor? Trata-se da utilização de Servlets e JSP!

O resultado foi a criação do **framework Struts**! Ele era uma implementação da Arquitetura MVC para desenvolvimento de páginas web dinâmicas! Bacana? Esse framework fez tanto sucesso que a Sun Microsystems junto com uma comunidade de desenvolvedores resolveu **criar uma especificação padronizada** baseada nesse framework, **denominado Java Server Faces (JSF)**.



Cuidado com as questões de prova que afirmam que se trata apenas de uma especificação – ele é tanto uma **especificação** quanto um **framework**<sup>1</sup>. Cuidado também com aquelas que afirmam que só trata de interfaces gráficas. Ele trata de componentes de interface com usuário – as interfaces gráficas (GUI) são um tipo de interface com o usuário (UI). Essa tecnologia consiste em dois aspectos:

Primeiro, uma API para representar componentes e gerenciar seus estados; manipular eventos; realizar validação server-side; converter dados; definir navegação de páginas; suportar internacionalização e acessibilidade; e prover extensibilidade. Segundo, taglibs (bibliotecas de tags) para adicionar componentes a páginas web e conectar componentes a objetos server-side.

Galera, ele provê um modelo de programação bem definido e robusto, além de fornecer diversas taglibs – inclusive o desenvolvedor pode criar sua própria taglib. Essas taglibs contêm manipuladores de tags que implementam os componentes de tags. Essas características facilitam significativamente o peso da construção e manutenção de Aplicações Web com interfaces de usuário server-side.

### O Modelo de Componentes JSF define três taglibs:

- **HTML**: possui componentes que representam diversos elementos HTML.
  - o Namespace: `xmlns:h="http://java.sun.com/jsf/html"`
- **CORE**: responsável por internacionalização, validação, conversão e outros.
  - o Namespace: `xmlns:f="http://java.sun.com/jsf/core"`
- **FACELETS**: fornece tags para criar templates para aplicações web.
  - o Namespace: `xmlns:ui="http://java.sun.com/jsf/facelets"`

Com o mínimo de esforço é possível criar uma página web; adicionar componentes em uma página ao adicionar tags de componentes; vincular componentes de uma página a dados server-side; conectar eventos gerados por componentes ao código da aplicação; salvar e restaurar o estado da aplicação além da vida da requisição do servidor; e reutilizar e estender componentes por meio de customização.

Podemos definir a Tecnologia JSF como uma tecnologia que nos permite criar Aplicações Web utilizando componentes visuais pré-prontos, de forma que o desenvolvedor não se preocupe com Javascript ou HTML. A funcionalidade fornecida por uma Aplicação JSF é similar a qualquer outra Aplicação Web. Ela possui as seguintes partes:

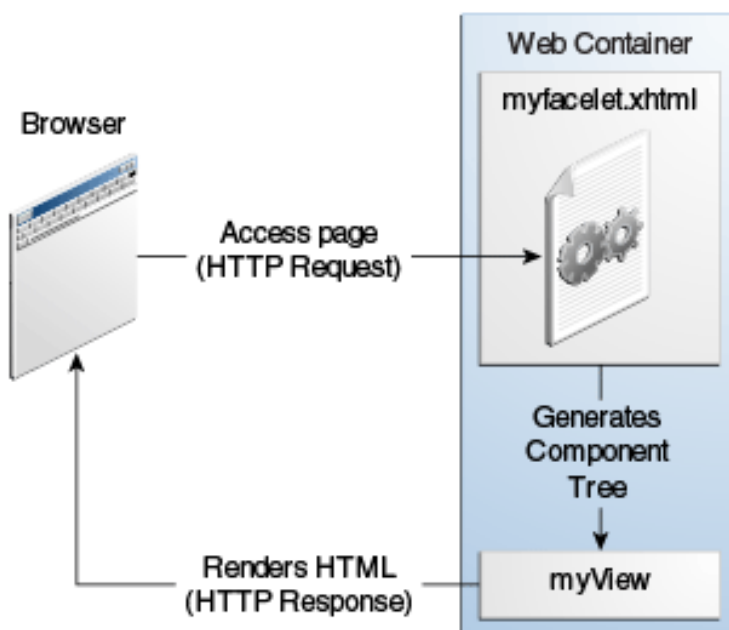
- Um conjunto de Páginas Web em que são colocados os componentes;

---

<sup>1</sup> As implementações da especificação mais famosas são Oracle Mojarra e o Apache MyFaces.

- Um conjunto de tags para adicionar componentes à página web;
- Um conjunto de Managed Beans (ou Beans Gerenciados);
- Um Descritor de Implantação Web (web.xml);
- Um ou mais arquivos de configuração (Ex: faces-config.xml);
- Um conjunto de objetos customizados (Ex: validadores, conversores, etc);
- Um conjunto de tags customizadas para representar objetos customizados;

Pessoal, o JSF oferece diversos validadores embutidos para validar seus Componentes UI – essa validação ocorre no lado do servidor. Eles podem ser invocados a partir de sua tag específica e podem validar o tamanho de um campo, tipo de entrada, range de um valor numérico, expressão regular, entre outros. É possível, inclusive, criar o seu próprio validador customizado.



Observem a imagem acima! Uma Página Web myfacelet.xhtml é construída utilizando tags de Componentes JSF. Essas tags são usadas para adicionar componentes à visão (myView), que é uma representação server-side da página. Além dos componentes, uma página web pode referenciar objetos como listeners, validadores, conversores, entre outros.

Em resposta a uma requisição do cliente, uma página web é renderizada por um contêiner web que implementa a tecnologia JSF! Uma de suas grandes vantagens é que ele oferece uma clara separação entre comportamento e apresentação. Ele mapeia solicitações HTTP para o tratamento de eventos específicos dos componentes e gerencia os componentes como objetos stateful no servidor.

Outro importante objetivo é aproveitar componentes e conceitos já familiares aos programadores, sem limitá-los a uma tecnologia de script ou a uma linguagem de marcação específicas. Isso possibilita a utilização de diferentes tecnologias de apresentação, a criação de componentes próprios a partir das classes de componentes, e a geração de saídas para diversos dispositivos (Ex: Celular, Tablet).

O JSF fornece uma maneira fácil e amigável para criar Aplicações Web por meio de, basicamente, **três atividades**:

- **Criar** uma Página Web (usando tags de componentes);
- **Desenvolver** Managed Beans;
- E **mapear** a instância FacesServlet.

Aqui vamos fazer uma pequena pausa! Professor, o que é um **Managed Bean**? São apenas **POJOs** com a annotation `@ManagedBeans`. Pensem no seguinte: meu sistema precisa escrever “Olá, pessoal” no navegador. Bem, esse texto não precisa de nenhuma informação, não acessa nada, é muito simples – basta colocá-lo diretamente na camada de visão e mostrá-lo!

E se eu tenho que dar um “Olá, X”, em que X é o nome da pessoa que acessou o sistema? Em outras palavras, se eu acessei, deve mostrar “Olá, Diego”; se o Messi acessou, deve mostrar “Olá, Messi”! Para tal, eu vou precisar acessar o banco de dados, buscar informações do sistema, talvez saber o horário de acesso, i.e., vou precisar interagir com o modelo, lógica de negócio ou componentes visuais.

Ora, nós prezamos pela separação de responsabilidades! Logo, esse código ficará em uma classe de modelo e, jamais, na visão. **Os Managed Beans** são os **objetos que intermediam** a comunicação entre **a visão e o modelo**. Eles são registrados no descritor de implantação (ou por meio de annotations) e tem seu **ciclo de vida controlado e gerenciado pelo próprio JSF**!

**As principais tarefas** de um **Managed Bean** (ou Backing Beans) é **fornecer dados** que serão exibidos nas telas; **receber dados** enviados nas requisições; **executar tarefas** de acordo com as ações dos usuários; **validar dados**. E o que seria a FacesServlet? É uma servlet que gerencia o ciclo de vida do processamento de requisições de aplicações web que estão utilizando JSF para construir a interface com o usuário.

Elas são responsáveis por receber as requisições da View, redirecioná-las para os Managed Beans do Model e respondê-las. Devemos configurá-la no descritor de implantação web.xml das aplicações web – ele faz a conexão entre Web Container e Web Application. Após isso, devemos configurar também o arquivo de configuração faces-config.xml, referente a uma aplicação específica que utiliza JSF.

Ele é responsável por descrever e configurar elementos e subelementos que compõem o projeto, tais como conversores, managed beans, validadores, fluxo da comunicação, configurações de localização e o mapeamento da navegação – ademais, ele faz a conexão entre View e Controller. Vamos resumir essa diferença entre esses dois arquivos?

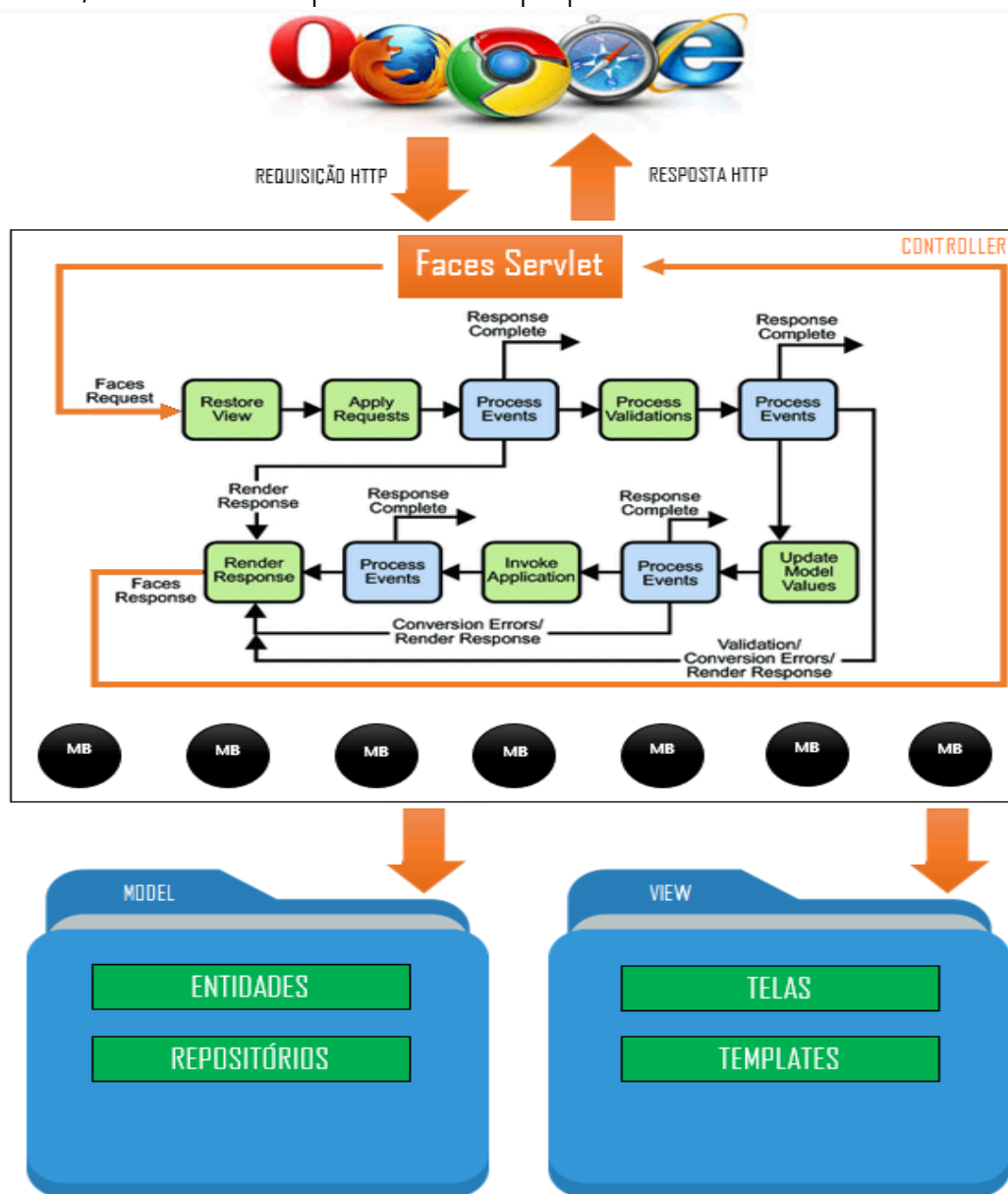
O faces-config.xml é mais específico, tratando de regras e mapeamento de navegação; definição de managed beans; configuração de detalhes de internacionalização; entre outros. Já o web.xml é mais genérico, tratando da especificação de detalhes de segurança; configuração de páginas de erro; mapeamento e declaração de servlets e filtros; configuração de parâmetros de inicialização; entre outros.

O faces-config.xml tem sido rapidamente substituído por annotations – novidade do JSF 2.0. Essa versão trouxe: suporte a facelets; utilização de templates para a aplicação; simplificação do desenvolvimento de componentes; suporte nativo a Ajax (f:ajax); navegação implícita e

condicional; suporte ao Método GET; adição de novos escopos (Flash e View); composição de componentes customizados; etc.

O JSF1 tinha os escopos Request (Default), Session e Application. A partir do JSF2, ganhamos o View, Flash, None e Custom. O @RequestScoped vive o tempo do ciclo de uma Requisição/Resposta HTTP; o @ViewScoped vive enquanto houver interação com a mesma view, i.e., enquanto persistir a mesma página; o @ApplicationScoped persiste toda a duração da aplicação web.

O @SessionScoped persiste o tempo que durar uma sessão, i.e., até invocar um método inválido ou o tempo acabar (lembrar de um carrinho de compras); o @FlashScoped dura um redirecionamento de página; o @NoneScoped indica que o escopo não está definido para a aplicação; por fim, o @CustomScoped é um escopo personalizado.



Agora eu queria falar uma curiosidade interessante! Existe um Padrão de Projeto Java EE chamado Front Controller. Nesse padrão, todas as requisições do usuário são recebidas pelo

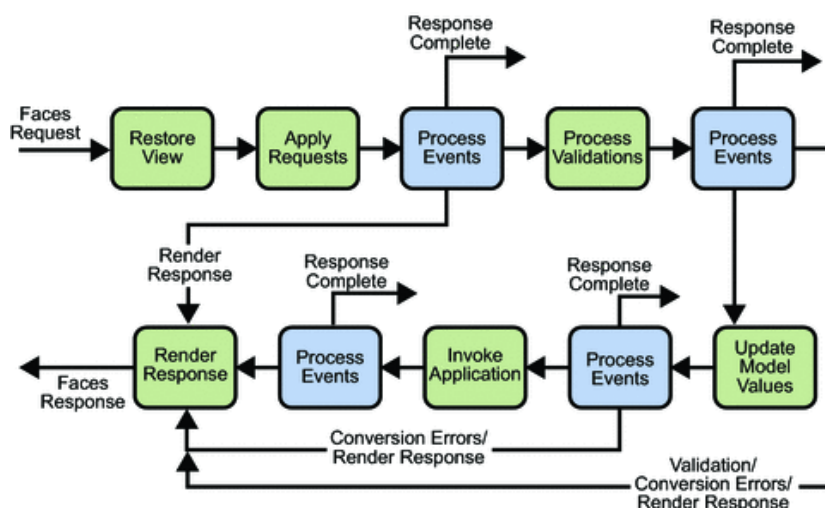
mesmo componente. Dessa forma, tarefas que devem ser realizadas em todas as requisições podem ser implementadas por esse componente – evitando repetição de código e facilitando a manutenção do sistema.

No JSF, esse componente é o FacesServlet! Como mostra a imagem da estrutura geral de uma Aplicação JSF, o processamento de uma requisição enviada por um navegador começa na FacesServlet. Observem que ela controla a execução das seis etapas do ciclo de vida, interagindo com o Model (Entidades e repositórios) e com as Views (Telas, Templates, etc).

Os Managed Beans estão à disposição da FacesServlet durante todo o processamento da requisição. Nas etapas Render Response e Restore View, a ela aciona os Managed Beans para recuperar os dados que devem ser usados na construção ou reconstrução da árvore de componentes. Na etapa Update Model, a FacesServlet armazena nos Managed Beans os dados já convertidos e validados.

Na etapa Invoke Application, a FacesServlet dispara um método em um Managed Bean responsável pelo processamento da regra de negócio correspondente à requisição atual. Todas as regras de negócio são implementadas no modelo, que também administra os dados da aplicação. Os Managed Beans acionam o modelo para executar regras de negócio, recuperar dados administrados pelo modelo, etc.

As telas da aplicação são definidas na camada de visão. A FacesServlet acessa essa camada toda vez que necessita construir ou reconstruir a árvore de componentes de uma determinada tela. Isso ocorre nas etapas Restore View e Render Response. Aliás, vamos ver agora rapidamente – porque não cai muuuuito em provas – o ciclo de vida do JSF! Ele é apresentado na imagem abaixo:



Pelo fato do framework JSF ser talvez uma evolução da linguagem JSP, o ciclo de vida do JSF é parecido com o do JSP. Por exemplo, quando o cliente faz uma Requisição HTTP para a página, o servidor responde com a página traduzida para HTML. Porém, ele é dividido em múltiplas fases, apresentando um modelo de componentes de interface com usuário mais sofisticado.

- **Restore View:** restauram-se os objetos e estruturas de dados que representam a visão. Claro, se essa for a primeira visita à página, deve-se criar a visão. Quando o JSF cria e renderiza uma

página JSF, ele cria objetos de interface com o usuário para cada componente da visão. Os componentes são armazenados em uma árvore de componentes e o estado da visão é salvo para requisições futuras.

- **Apply Request Values:** qualquer dado que for enviado como parte da requisição é passado para os componentes apropriados. Essas visões atualizam seus estados com os valores dos dados. Dados podem vir de formulários, cookies enviados com a requisição ou por meio de cabeçalhos da requisição. Alguns dados são validados e, se houver erro, são adicionados à FacesServlet.
- **Process Validation:** os dados que foram submetidos com o formulário são validados (se já não o foram anteriormente). Assim como na fase anterior, isso ainda não atualiza os objetos de negócio na aplicação. Isso ocorre porque, se a Aplicação atualizar os objetos de negócio junto com a validação dos dados e uma parte da validação falhar, o modelo será atualizado com um estado inválido.
- **Update Model Values:** após todas essas validações terminarem, os objetos de negócio que criam a aplicação são atualizados com os dados validados da requisição. Ademais, se qualquer um dos dados precisar ser convertido em um formato diferente para atualizar o modelo (Ex: String para Data), a conversão ocorrerá nessa fase.
- **Invoke Application:** durante essa fase, os métodos de ação de qualquer botão ou link que foi ativado serão chamados. Além disso, todos os eventos que foram gerados durante as fases anteriores e que ainda não tenham sido manipulados são passados para a Aplicação Web para que ela possa concluir qualquer outro processamento da requisição que seja necessário.
- **Render Response:** os Componentes UI de resposta são renderizados e a resposta é enviada para o cliente. O estado dos componentes é salvo de modo que a árvore de componente possa ser restaurada quando o cliente enviar outra requisição. Em suma, essa fase renderizará a página de resposta requisitada pelo usuário.

Lembrando que FacesContext (javax.faces.context) é o objeto utilizado para representar todas as informações de contexto associadas ao processamento da requisição de entrada e à criação da resposta correspondente. Ela é criada pela FacesServlet, que é executada antes do início do ciclo de vida de processamento de requisições e é responsável por gerenciar a execução das etapas do ciclo de vida.

Vamos falar um pouquinho sobre Component Binding! O que é isso, professor? Cara, essa é uma nova característica da tecnologia JSF que permite associar componentes de uma view e controlar todos os aspectos desse componente. Como assim? Algumas vezes, nós temos um componente visual que nos oferece alguma informação (Ex: um mapa em que o usuário escolhe estado).

Bem, em geral, nós necessitamos apenas do valor, i.e., o usuário escolheu 'DF'. O Component Binding permite que nós tenhamos acesso ao componente como um todo. Para que? Nós, eventualmente, podemos querer manipular o componente dinamicamente, por exemplo. Assim, é possível acessar métodos do componente – se você quiser, pode até mudar seu comportamento.

## JSF : Facelets

Pessoal, tem uma característica do JSF que é extremamente importante: **Facelets**! Trata-se de uma **linguagem de declaração de página** poderosa, apesar de leve. Antigamente, utiliza-se a tecnologia JSP como camada de visão do JSF, porém ele não suporta todas as características disponíveis na Plataforma Java EE – sendo considerada obsoleta para JSF!

Facelets é uma parte da especificação JSF e também a tecnologia de apresentação preferida para construir aplicações JSF – substituindo JSP. Ela suporta todos os componentes de UI do JSF e constrói Árvores de Componentes; e Views (utilizando Templates HTML). Um tipo especial de template são os Componentes Compostos, que agem como um componente. Ademais, é bom destacar algumas **características**:

- Utilização de XHTML para criação de Páginas Web;
- Suporte a Facelets Tag Libraries (além do JSFTL e JSTL);
- Suporte a Linguagens de Expressão (Expression Languages);
- Suporte a templates para componentes e páginas.

Em suma, a **utilização de Facelets reduz o tempo e esforço** gastos no desenvolvimento e implantação. Em geral, Facelets Views são criadas com Páginas HTML e XHTML (criadas em conformidade com Transation DTD). Além disso, elas utilizam linguagens de expressão para referenciar propriedades e Managed Beans. Para desenvolver uma Aplicação Facelets simples, é necessário:

- Desenvolver um Managed Bean;
- Criar páginas usando tags de componentes;
- Definir a navegação das páginas;
- Mapear a instância javax.faces.webapp.FacesServlet;
- Adicionar declarações de Managed Beans.

## JSF : Filtros

Em JSF, um **Filtro** (do inglês, Filter) é um objeto capaz de **realizar tarefas de filtragem** tanto na requisição de um recurso (Servlet ou conteúdo estático), ou na resposta desse recurso, ou ambos – para tal, eles utilizam o método doFilter. Todo Filtro possui acesso a um objeto FilterConfig, do qual ele pode obter seus parâmetros de inicialização; e a uma ServletContext, do qual ele pode carregar recursos.

Os Filtros são configurados nos descritores de implantação de uma Aplicação Web. Ele possibilita o gerenciamento de todas as Requisições HTTP do Servidor, capaz de filtrar o endereço que está sendo acessado. Dessa forma, quando um usuário acessar uma determinada URL proibida, pode-se imediatamente redirecioná-lo para outro endereço, antes que a resposta seja dada ao cliente.

Para tal, deve-se implementar a interface javax.servlet.Filter! Existem dezenas de aplicações para filtros, além da mostrada acima. Podemos ter filtros de autenticação; filtros de log e auditoria;

filtros de conversão de imagens; filtros de compressão de dados; filtros de criptografia; filtros de tokenização; filtros XSLT; filtros que acionam eventos de acesso a recursos, entre outros.

## QUESTÕES COMENTADAS - JSF - MULTIBANCAS

1. (FCC – 2013 – TRT/12 – Analista de Sistemas) Considere as instruções abaixo encontradas em um arquivo de uma aplicação que utiliza JSF:

```
<managed-bean>  
<managed-bean-name>func</managed-bean-name>  
<managed-bean-class>bean.Funcionario</managed-bean-class>  
<managed-bean-scope>session</managed-bean-scope>  
</managed-bean>
```

Essas instruções indicam a existência de um bean gerenciado (classe Funcionario.java) no pacote bean que poderá ser referenciado nas páginas JSP por meio da palavra func. O arquivo correto no qual essas instruções são colocadas é o:

- a) context.xml.
- b) web-inf.xml.
- c) web.xml.
- d) faces-config.xml.
- e) config-bean.xml.

### Comentários:

Elas são responsáveis por receber as requisições da View, redirecioná-las para os Managed Beans do Model e respondê-las. Devemos configurá-la no descritor de implantação web.xml das aplicações web – ele faz a conexão entre Web Container e Web Application. Após isso, devemos configurar também o arquivo de configuração faces-config.xml, referente a uma aplicação específica que utiliza JSF.

Conforme vimos em aula, o responsável é o faces-config.xml. **Gabarito: D**

2. (CESPE - 2009 - SECONT-ES - Auditor do Estado – Tecnologia da Informação) O JSF é um framework web embasado em interface gráfica, capaz de renderizar componentes e manipular eventos em aplicações web no padrão Java EE, no qual os componentes JSF são orientados a eventos. O JSF fornece, ainda, mecanismos para conversão, validação, execução de lógica de negócios e controle de navegação.

### Comentários:

Primeiro, uma API para representar componentes e gerenciar seus estados; manipular eventos; realizar validação server-side; converter dados; definir navegação de páginas; suportar

internacionalização e acessibilidade; e prover extensibilidade. Segundo, taglibs (bibliotecas de tags) para adicionar componentes a páginas web e conectar componentes a objetos server-side.

Conforme vimos em aula, a questão está perfeita! **Gabarito: C**

3. (FCC - 2012 - TJ-PE - Programador de computador) Em uma aplicação que utiliza JSF, para configurar o fluxo de comunicação presente na servlet de controle, é utilizado um arquivo de configuração:

- a) webfaces.xml.
- b) actionform.xml.
- c) faces-config.xml.
- d) webcontext.xml.
- e) serverconfig.xml.

#### Comentários:

Elas são responsáveis por receber as requisições da View, redirecioná-las para os Managed Beans do Model e respondê-las. Devemos configurá-la no descritor de implantação web.xml das aplicações web – ele faz a conexão entre Web Container e Web Application. Após isso, devemos configurar também o arquivo de configuração faces-config.xml, referente a uma aplicação específica que utiliza JSF.

Ele é responsável por descrever e configurar elementos e subelementos que compõem o projeto, tais como conversores, managed beans, validadores, fluxo da comunicação, configurações de localização e o mapeamento da navegação – ademais, ele faz a conexão entre View e Controller. Vamos resumir essa diferença entre esses dois arquivos?

Conforme vimos em aula, trata-se do faces-config.xml. **Gabarito: C**

4. (CESPE - 2010 - TRE-BA - Analista Judiciário - Análise de Sistemas) Entre os itens que o padrão Java Server Faces (JSF) utiliza, estão os componentes, os eventos e a navegabilidade.

#### Comentários:

Primeiro, uma API para representar componentes e gerenciar seus estados; manipular eventos; realizar validação server-side; converter dados; definir navegação de páginas; suportar internacionalização e acessibilidade; e prover extensibilidade. Segundo, taglibs (bibliotecas de tags) para adicionar componentes a páginas web e conectar componentes a objetos server-side.

Conforme vimos em aula, a questão está perfeita! **Gabarito: C**

5. (FCC – 2013 – TRT/9 – Analista de Sistemas) Uma aplicação utilizando o framework JSF e a IDE NetBeans gera automaticamente dois componentes essenciais assim descritos:

I. É responsável por receber requisições dos componentes View do MVC, redirecioná-las para os beans gerenciados (managed beans) do componente Model do MVC e responder a essas requisições.

II. É o arquivo principal de configuração de uma aplicação web que utiliza o framework JSF. É responsável por descrever os elementos e sub-elementos que compõem o projeto, tais como as regras de navegação, beans gerenciados, configurações de localização etc.

As descrições I e II referem-se, respectivamente, aos componentes:

- a) servlet Controller.java e ao arquivo faces\_config.xml
- b) FaceletServlet e ao arquivo web\_config.xml.
- c) FacesServlet e ao arquivo faces-config.xml.
- d) servlet Controller e ao arquivo web-config.xml.
- e) servlet Facelet e ao arquivo web.xml.

#### Comentários:

Elas são responsáveis por receber as requisições da View, redirecioná-las para os Managed Beans do Model e respondê-las. Devemos configurá-la no descritor de implantação web.xml das aplicações web – ele faz a conexão entre Web Container e Web Application. Após isso, devemos configurar também o arquivo de configuração faces-config.xml, referente a uma aplicação específica que utiliza JSF.

Ele é responsável por descrever e configurar elementos e subelementos que compõem o projeto, tais como conversores, managed beans, validadores, fluxo da comunicação, configurações de localização e o mapeamento da navegação – ademais, ele faz a conexão entre View e Controller. Vamos resumir essa diferença entre esses dois arquivos?

Conforme vimos em aula, o primeiro é o FacesServlet e o segundo faces-config.xml. **Gabarito: C**

6. (CESPE - 2012 - ANAC - Analista Administrativo - Área 4) A validação de dados de um componente pode ser uma das funções de um backing bean, em uma aplicação JSF.

#### Comentários:

As principais tarefas de um Managed Bean (ou Backing Beans) é fornecer dados que serão exibidos nas telas; receber dados enviados nas requisições; executar tarefas de acordo com as ações dos usuários; validar dados. E o que seria a FacesServlet? É uma servlet que gerencia o ciclo de vida do processamento de requisições de aplicações web que estão utilizando JSF para construir a interface com o usuário.

Conforme vimos em aula, Backing Beans são Managed Beans, e essa pode ser uma de suas funções. **Gabarito: C**

7. (FCC – 2012 – TST – Analista de Sistemas) O framework JavaServer Faces (JSF) é utilizado no desenvolvimento de aplicações web que utiliza o design pattern MVC. O JSF:

- a) disponibiliza controles pré-construídos e código para manipular eventos, estimulando o uso de código Java convencional no componente View do MVC.
- b) recebe requisições dos componentes da View do MVC, através do servlet FaveServerServlet.
- c) armazena os mapeamentos das ações e regras de navegação em projetos JSF nos arquivos WEB-INF.xml e FACES-CONFIG.xml.
- d) possui bibliotecas que suportam Ajax (Asynchronous JavaScript And XML).
- e) provê um conjunto de tags limitado para criar somente páginas HTML/XHTML.

#### Comentários:

Ora, nós prezamos pela separação de responsabilidades! Logo, esse código ficará em uma classe de modelo e, jamais, na visão. Os Managed Beans são os objetos que intermediam a comunicação entre a visão e o modelo. Eles são registrados no descritor de implantação (ou por meio de annotations) e tem seu ciclo de vida controlado e gerenciado pelo próprio JSF!

(a) Conforme vimos em aula, está incorreto, i.e., é desestimulado colocar código convencional (regras de negócio) na View, mas – sim – no Model.

Elas são responsáveis por receber as requisições da View, redirecioná-las para os Managed Beans do Model e respondê-las. Devemos configurá-la no descritor de implantação web.xml das aplicações web – ele faz a conexão entre Web Container e Web Application. Após isso, devemos configurar também o arquivo de configuração faces-config.xml, referente a uma aplicação específica que utiliza JSF.

(b) Conforme vimos em aula, de fato recebe requisições dos componentes da View, no entanto o nome da servlet é FacesServlet.

Elas são responsáveis por receber as requisições da View, redirecioná-las para os Managed Beans do Model e respondê-las. Devemos configurá-la no descritor de implantação web.xml das aplicações web – ele faz a conexão entre Web Container e Web Application. Após isso, devemos configurar também o arquivo de configuração faces-config.xml, referente a uma aplicação específica que utiliza JSF.

Ele é responsável por descrever e configurar elementos e subelementos que compõem o projeto, tais como conversores, managed beans, validadores, fluxo da comunicação, configurações de localização e o mapeamento da navegação – ademais, ele faz a conexão entre View e Controller. Vamos resumir essa diferença entre esses dois arquivos?

(c) Conforme vimos em aula, o mapeamento de ações e regras de navegação é responsabilidade somente do faces-config.xml.

O faces-config.xml têm sido rapidamente substituído por annotations – novidade do JSF 2.0. Essa versão trouxe: suporte a facelets; utilização de templates para a aplicação; simplificação do desenvolvimento de componentes; suporte nativo a Ajax (f:ajax); navegação implícita e condicional; suporte ao Método GET; adição de novos escopos (Flash e View); composição de componentes customizados; etc.

(d) Conforme vimos em aula, a questão está perfeita! O Ajax não só é suportado, esse suporte é nativo.

Galera, ele provê um modelo de programação bem definido e robusto, além de fornecer diversas taglibs – inclusive o desenvolver pode criar sua própria taglib. Essas taglibs contêm manipuladores de tags que implementam os componentes de tags. Essas características facilitam significativamente o peso da construção e manutenção de Aplicações Web com interfaces de usuário server-side.

(e) Conforme vimos em aula, não se trata de um conjunto limitado de tags. É possível criar suas próprias tags!

**Gabarito: D**

8. (CESPE - 2013 - SERPRO - Analista - Desenvolvimento de Sistemas) O JSF provê uma linguagem de expressão exclusiva para acesso a objetos armazenados em bancos de dados.

**Comentários:**

Em suma, a utilização de Facelets reduz o tempo e esforço gastos no desenvolvimento e implantação. Em geral, Facelets Views são criadas com Páginas HTML e XHTML (criadas em

conformidade com Transationl DTD). Além disso, elas utilizam linguagens de expressão para referenciar propriedades e Managed Beans. Para desenvolver uma Aplicação Facelets simples, é necessário:

Conforme vimos em aula, ela serve para referenciar propriedades e Managed Beans.

**Gabarito: E**

9. (FCC – 2012 – TST – Analista de Sistemas) Para criar as páginas XHTML de uma aplicação JSF é possível utilizar um conjunto de bibliotecas de tags JSF. Algumas dessas bibliotecas são HTML, Core e Facelets. Considere os fragmentos de códigos abaixo, que utilizam tags dessas bibliotecas:

Fragmento de código I:

```
<h1>  
<ui:insert name="titulo">Título </ui:insert>  
</h1>
```

Fragmento de código II:

```
<h:inputText id="usuário" value="#{usuarioBean.usuario.  
nome}"/>
```

Fragmento de código III:

```
<h:selectOneMenu id="lista">  
<f:selectItems  
value="#{optionBean.optionList}"></f:selectItem>  
</h:selectOneMenu>
```

A correlação correta entre o fragmento de código e a biblioteca de tags utilizada é:

- a) I-Facelets, II-HTML e III-Core.
- b) I-Core, II-Facelets, e III-HTML.
- c) I-HTML, II-Core, e III-Facelets.
- d) I-HTML, II-Facelets, e III-HTML.
- e) I-HTML, II-HTML, e III-Core.

**Comentários:**

O Modelo de Componentes JSF define três taglibs:

- ☞ HTML: possui componentes que representam diversos elementos HTML.
  - o Namespace: xmlns:h="http://java.sun.com/jsf/html"
- ☞ CORE: responsável por internacionalização, validação, conversão e outros.
  - o Namespace: xmlns:f="http://java.sun.com/jsf/core"
- ☞ FACELETS: fornece tags para criar templates para aplicações web.
  - o Namespace: xmlns:ui="http://java.sun.com/jsf/facelets"

Conforme vimos em aula, temos que olhar os namespaces. Dessa forma, o primeiro fragmento é referente a FACELETS; o segundo fragmento é referente a HTML; e o terceiro referente a CORE.

**Gabarito: A**

10.(CESPE - 2010 – TCU – Analista de Sistemas) No desenvolvimento de conteúdos para apresentação, o uso de facelets traz vantagens em relação ao uso de JSP. Uma delas é a maior modularidade, com o uso de templates e componentes compostos (composite).

**Comentários:**

Facelets é uma parte da especificação JSF e também a tecnologia de apresentação preferida para construir aplicações JSF – substituindo JSP. Ela suporta todos os componentes de UI do JSF e constrói Árvores de Componentes; e Views (utilizando Templates HTML). Um tipo especial de template são os Componentes Compostos, que agem como um componente. Ademais, é bom destacar algumas características:

Conforme vimos em aula, ela faz uso de Templates e Componentes Compostos, que aumentam a modularidade e a reusabilidade **Gabarito: C**

11.(FCC - 2012 - TJ-PE - Analista Judiciário - Análise de Sistemas) No JSF, o componente Controller do MVC é composto por uma classe servlet, por arquivos de configuração e por um conjunto de manipuladores de ações e observadores de eventos. Essa servlet é chamada de:

- a) ControllerServlet.
- b) Facelet.
- c) HttpServlet.
- d) FacesConfig.
- e) FacesServlet.

**Comentários:**

As principais tarefas de um Managed Bean (ou Backing Beans) é fornecer dados que serão exibidos nas telas; receber dados enviados nas requisições; executar tarefas de acordo com as ações dos usuários; validar dados. E o que seria a FacesServlet? É uma servlet que gerencia o ciclo de vida do processamento de requisições de aplicações web que estão utilizando JSF para construir a interface com o usuário.

Conforme vimos em aula, trata-se da FacesServlet. **Gabarito: E**

- 12.(CESPE - 2012 – TJ/AL – Analista de Sistemas – B) Em um aplicativo Facelets, a tag f:ajax adiciona funcionalidades Ajax que necessitam de adicionais de codificação e configuração para as componentes de interface do usuário.

#### Comentários:

O faces-config.xml têm sido rapidamente substituído por annotations – novidade do JSF 2.0. Essa versão trouxe: suporte a facelets; utilização de templates para a aplicação; simplificação do desenvolvimento de componentes; suporte nativo a Ajax (f:ajax); navegação implícita e condicional; suporte ao Método GET; adição de novos escopos (Flash e View); composição de componentes customizados; etc.

Conforme vimos em aula, o suporte é nativo. Logo, não é necessário codificação e configuração adicionais:

“By using the f:ajax tag along with another standard component in a Facelets application. This method adds Ajax functionality to any UI component without additional coding and configuration”. **Gabarito: E**

- 13.(FCC - 2012 - TRE-CE - Analista Judiciário - Análise de Sistemas) No ciclo de vida do Java Server Faces trata-se da fase na qual o componente deve primeiro ser criado ou recuperado a partir do FacesContext, seguido por seus valores, que são geralmente recuperados dos parâmetros de request e, eventualmente, dos cabeçalhos ou cookies gerados. Trata-se da fase:

- a) Restore View.
- b) Apply Request Values.
- c) Process Validation.
- d) Update Model Values.
- e) Invoke Application.

#### Comentários:

- ☞ Apply Request Values: qualquer dado que for enviado como parte da requisição é passado para os componentes apropriados. Essas visões atualizam seus estados com os valores dos

dados. Dados podem vir de formulários, cookies enviados com a requisição ou por meio de cabeçalhos da requisição. Alguns dados são validados e, se houver erro, são adicionados à FacesServlet.

Conforme vimos em aula, trata-se da fase Apply Request Values. **Gabarito: B**

14.(CESPE - 2012 – TJ/RO – Analista de Sistemas – E) JNDI, parte do projeto de JSF, utiliza XHTML como tecnologia de apresentação dos dados, possibilitando a separação entre as camadas de negócio e de controle.

**Comentários:**

Em suma, a utilização de Facelets reduz o tempo e esforço gastos no desenvolvimento e implantação. Em geral, Facelets Views são criadas com Páginas HTML e XHTML (criadas em conformidade com Transationl DTD). Além disso, elas utilizam linguagens de expressão para referenciar propriedades e Managed Beans. Para desenvolver uma Aplicação Facelets simples, é necessário:

Conforme vimos em aula, a questão trata – na verdade – das Facelets. **Gabarito: E**

15.(FCC - 2012 - TRT - 11ª Região (AM) - Técnico Judiciário - Tecnologia da Informação) Sobre o framework JavaServer Faces é correto afirmar:

- a) A grande limitação do JSF é a dificuldade de integração com outros frameworks como Spring, JPA e EJB.
- b) Expression Language (EL) é a linguagem utilizada para apresentação de conteúdo em aplicações que utilizam JSF. Sua principal limitação é a impossibilidade de acessar valores e métodos em beans gerenciados.
- c) Facelets é uma parte da especificação JSF e também a tecnologia para implementar as regras de negócio em aplicações que utilizam JSF.
- d) Disponibiliza as bibliotecas de tags core e html para criar as páginas que compõem a interface do usuário.
- e) Define uma única forma para realizar a validação de dados em formulários JSP, por meio da implementação de uma classe de validação que estende a interface Validator.

**Comentários:**

(a) Na verdade, está na Camada Web, facilmente integrável com Spring, JPA e EJB;

- (b) Na verdade, é facilmente possível;
- (c) Na verdade, Facelets tratam da camada de visão e, não, de regras de negócio;
- (d) Perfeito, basta utilizar várias taglibs prontas ou criá-las;
- (e) Na verdade, existem diversas formas.

**Gabarito: D**

16.(CESPE - 2013 – CPRM – Analista de Sistemas) Facelets são utilizadas para desenvolver visões (views) JavaServer Faces (JSF) com linguagem HTML e XHTML, em conformidade com a transitional document type definition, sendo, ainda, compatível com a biblioteca de tag JSF.

**Comentários:**

Em suma, a utilização de Facelets reduz o tempo e esforço gastos no desenvolvimento e implantação. Em geral, Facelets Views são criadas com Páginas HTML e XHTML (criadas em conformidade com Transitional DTD). Além disso, elas utilizam linguagens de expressão para referenciar propriedades e Managed Beans. Para desenvolver uma Aplicação Facelets simples, é necessário:

Conforme vimos em aula, a questão está perfeita! **Gabarito: C**

17.(FCC - 2011 - TRE-AP - Técnico Judiciário - Programação de Sistemas) O JSF extrai todos os valores digitados pelo usuário e guarda esse valor nos seus respectivos componentes. Se o valor digitado não coincidir com o componente, um erro vai ser adicionado na classe FacesContext e será mostrado na fase Render Response Phase.

No ciclo de vida do JSF trata-se de um evento típico da fase:

- a) Process Validations Phase.
- b) Restore View Phase.
- c) Apply Request Values Phase.
- d) Update Model Values Phase.
- e) Invoke Application Phase.

**Comentários:**

☞ Apply Request Values: qualquer dado que for enviado como parte da requisição é passado para os componentes apropriados. Essas visões atualizam seus estados com os valores dos dados. Dados podem vir de formulários, cookies enviados com a requisição ou por meio de cabeçalhos da requisição. Alguns dados são validados e, se houver erro, são adicionados à FacesServlet.

Conforme vimos em aula, trata-se da fase Apply Request Values. **Gabarito: C**

18.(CESPE - 2013 – INPI - Analista Judiciário - Análise de Sistemas) Quando registrado em JSF 2 (Java Server Faces), um managed bean permanece no escopo de session.

Comentários:

O JSF1 tinha os escopos Request (Default), Session e Application. A partir do JSF2, ganhamos o View, Flash, None e Custom. O @RequestScoped vive o tempo do ciclo de uma Requisição/Resposta HTTP; o @ViewScoped vive enquanto houver interação com a mesma view, i.e., enquanto persistir a mesma página; o @ApplicationScoped persiste toda a duração da aplicação web.

No JSF2, o Escopo Request continua sendo o padrão (default)! **Gabarito: E**

19.(CESPE - 2010 – TCU – Analista de Sistemas) Para suportar a construção de aplicações com Ajax e JSF, recomenda-se aos desenvolvedores de páginas que usem a tag <f:ajax>, relacionada ao processamento de pedidos http assíncronos.

Comentários:

O faces-config.xml têm sido rapidamente substituído por annotations – novidade do JSF 2.0. Essa versão trouxe: suporte a facelets; utilização de templates para a aplicação; simplificação do desenvolvimento de componentes; suporte nativo a Ajax (f:ajax); navegação implícita e condicional; suporte ao Método GET; adição de novos escopos (Flash e View); composição de componentes customizados; etc.

Conforme vimos em aula, a questão está perfeita! **Gabarito: C**

20.(FCC - 2011 - TRE-RN - Técnico Judiciário - Programação de Sistemas) No ciclo de vida do JSF copiar os parâmetros de requisição para valores submetidos pelos componentes, é a tarefa típica da fase:

- a) Restaurar Visão (Restore view).
- b) Invocar aplicação (Invoke application).
- c) Aplicar valores de requisição (Apply request values).
- d) Processar validações (Process validation).

e) Atualizar valores do modelo (Update model values).

#### Comentários:

📖 Apply Request Values: qualquer dado que for enviado como parte da requisição é passado para os componentes apropriados. Essas visões atualizam seus estados com os valores dos dados. Dados podem vir de formulários, cookies enviados com a requisição ou por meio de cabeçalhos da requisição. Alguns dados são validados e, se houver erro, são adicionados à FacesServlet.

Conforme vimos em aula, trata-se da fase Apply Request Values. **Gabarito: C**

21.(CESPE - 2010 – MPU – Analista de Sistemas) Uma aplicação web deve prover mecanismos de validação de dados. O JSF fornece vários validadores de dados padrões que podem ser utilizados no lado do cliente (client-side).

#### Comentários:

Pessoal, o JSF oferece diversos validadores embutidos para validar seus Componentes UI – essa validação ocorre no lado do servidor. Eles podem ser invocados a partir de sua tag específica e podem validar o tamanho de um campo, tipo de entrada, range de um valor numérico, expressão regular, entre outros. É possível, inclusive, criar o seu próprio validador customizado.

Conforme vimos em aula, ocorre do lado servidor! **Gabarito: E**

22.(FCC - 2010 - TRT - 22ª Região (PI) - Técnico Judiciário - Tecnologia da Informação) É um framework MVC utilizado no desenvolvimento de aplicações para a internet de forma visual, que utiliza o recurso de arrastar e soltar os componentes na tela para definir suas propriedades:

- a) Enterprise JavaBeans.
- b) JavaServer Faces.
- c) Java 2 Enterprise Edition.
- d) Servlets.
- e) Java Server Pages.

#### Comentários:

O resultado foi a criação do framework Struts! Ele era uma implementação da Arquitetura MVC para desenvolvimento de páginas web dinâmicas! Bacana? Esse framework fez tanto sucesso que a Sun Microsystems junto com uma comunidade de desenvolvedores resolveu criar uma especificação padronizada baseada nesse framework, denominado Java Server Faces (JSF).

Conforme vimos em aula, trata-se do JSF! **Gabarito: B**

23.(CESPE - 2015 – TCU - Analista de Sistemas) A partir da interpretação do trecho JSF (JavaServer Faces), versão 2, no código a seguir, verifica-se que uma providência válida é configurar o managed-bean clientePage no arquivo faces-config.xml.

```
<f:view>
<h:form id="clienteForm">
<h:outputLabel for="informeNome" value="Informe Nome"/>
<h:inputText id="informeNome" value="#{clientePage.nome}"/>
<h:commandButton value="Nome do Cliente"
action="#{clientePage.cliente}"/>
</h:form>
</f:view>
```

#### Comentários:

O faces-config.xml é mais específico, tratando de regras e mapeamento de navegação; definição de managed beans; configuração de detalhes de internacionalização; entre outros. Já o web.xml é mais genérico, tratando da especificação de detalhes de segurança; configuração de páginas de erro; mapeamento e declaração de servlets e filtros; configuração de parâmetros de inicialização; entre outros.

Conforme vimos em aula, a questão está perfeita! É possível declarar managed beans por meio de anotações ou por meio do arquivo de configuração faces-config.xml. **Gabarito: C**

24.(CESPE - 2014 - TJ-SE - Analista Judiciário - Análise de Sistemas) Antes de uma aplicação web desenvolvida nos moldes da JSF executar sua primeira página web, uma instância FacesServlet é executada, a fim de gerenciar as requisições dessa aplicação.

#### Comentários:

Lembrando que FacesContext (javax.faces.context) é o objeto utilizado para representar todas as informações de contexto associadas ao processamento da requisição de entrada e à criação da

resposta correspondente. Ela é criada pela FacesServlet, que é executada antes do início do ciclo de vida de processamento de requisições e é responsável por gerenciar a execução das etapas do ciclo de vida.

Conforme vimos em aula, a questão está perfeita! **Gabarito: C**

25.(FGV - 2013 – ALEMA - Analista de Sistemas) Com relação à especificação Java Server Faces (JSF), assinale V para a afirmativa verdadeira e F para a falsa.

( ) Visa substituir a especificação Java Server Pages.

( ) Java Server Faces são usadas como uma fachada para Servlets e Java Server Pages.

( ) Define um framework MVC (Model View Controller) para aplicações Web.

As afirmativas são, respectivamente,

- a) F, F e V.
- b) F, V e V.
- c) V, F e F.
- d) V, V e F.
- e) F, V e F.

Comentários:

Facelets é uma parte da especificação JSF e também a tecnologia de apresentação preferida para construir aplicações JSF – substituindo JSP. Ela suporta todos os componentes de UI do JSF e constrói Árvores de Componentes; e Views (utilizando Templates HTML). Um tipo especial de template são os Componentes Compostos, que agem como um componente. Ademais, é bom destacar algumas características:

(F) Conforme vimos em aula, Facelets vieram para substituir JSP e, não, JSF!

Essa eu vou explicar melhor, porque sempre me perguntar: se eu afirmar que o Facebook visava substituir o MSN, isso é certo ou errado? Errado, ele está em um contexto muito maior; uma de suas funcionalidades (chat) veio de fato a substituir o MSN! JSF é um framework imenso, com

centenas de funcionalidades. Ora, eu não posso afirmar que ele visa substituir JSP! Na verdade, uma de suas funcionalidades vem substituindo JSP, mas não o JSF como um todo.

(F) Não, esse item não faz nenhum sentido.

O resultado foi a criação do framework Struts! Ele era uma implementação da Arquitetura MVC para desenvolvimento de páginas web dinâmicas! Bacana? Esse framework fez tanto sucesso que a Sun Microsystems junto com uma comunidade de desenvolvedores resolveu criar uma especificação padronizada baseada nesse framework, denominado Java Server Faces (JSF).

(V) Perfeito, implementa o padrão MVC! **Gabarito: A**

26.(CESPE - 2014 - TJ-SE - Analista Judiciário - Análise de Sistemas) Em aplicações web nos padrões da JSF, é possível utilizar recursos Ajax para criar páginas dinâmicas, como, por exemplo, por meio da tag f:ajax, conforme apresentado na sintaxe abaixo.

```
<h:inputText value="#{bean.message}">  
  <f:ajax />  
</h:inputText>
```

Comentários:

O faces-config.xml têm sido rapidamente substituído por annotations – novidade do JSF 2.0. Essa versão trouxe: suporte a facelets; utilização de templates para a aplicação; simplificação do desenvolvimento de componentes; suporte nativo a Ajax (f:ajax); navegação implícita e condicional; suporte ao Método GET; adição de novos escopos (Flash e View); composição de componentes customizados; etc.

Conforme vimos em aula, a questão está perfeita! **Gabarito: C**

27.(CESPE - 2014 - TJ-SE - Analista Judiciário - Análise de Sistemas) É possível utilizar XHTML no desenvolvimento de facelets para criar páginas web compatíveis com a JSF (JavaServer Faces) para apresentação dos dados. Na versão Java EE 7, essa forma de apresentação é mais indicada que a JSP (JavaServer Pages), uma vez que esta não suporta todos os novos recursos da versão Java EE 7.

Comentários:

Pessoal, tem uma característica do JSF que é extremamente importante: Facelets! Trata-se de uma linguagem de declaração de página poderosa, apesar de leve. Antigamente, utiliza-se a tecnologia JSP como camada de visão do JSF, porém ele não suporta todas as características disponíveis na Plataforma Java EE – sendo considerada obsoleta para JSF!

Conforme vimos em aula, a questão está perfeita! **Gabarito: C**

## LISTA DE QUESTÕES – JSF - MULTIBANCAS

1. (FCC – 2013 – TRT/12 – Analista de Sistemas) Considere as instruções abaixo encontradas em um arquivo de uma aplicação que utiliza JSF:

```
<managed-bean>  
<managed-bean-name>func</managed-bean-name>  
<managed-bean-class>bean.Funcionario</managed-bean-class>  
<managed-bean-scope>session</managed-bean-scope>  
</managed-bean>
```

Essas instruções indicam a existência de um bean gerenciado (classe Funcionario.java) no pacote bean que poderá ser referenciado nas páginas JSP por meio da palavra func. O arquivo correto no qual essas instruções são colocadas é o:

- a) context.xml.
  - b) web-inf.xml.
  - c) web.xml.
  - d) faces-config.xml.
  - e) config-bean.xml.
2. (CESPE - 2009 - SECONT-ES - Auditor do Estado – Tecnologia da Informação) O JSF é um framework web embasado em interface gráfica, capaz de renderizar componentes e manipular eventos em aplicações web no padrão Java EE, no qual os componentes JSF são orientados a eventos. O JSF fornece, ainda, mecanismos para conversão, validação, execução de lógica de negócios e controle de navegação.
3. (FCC - 2012 - TJ-PE - Programador de computador) Em uma aplicação que utiliza JSF, para configurar o fluxo de comunicação presente na servlet de controle, é utilizado um arquivo de configuração:
- a) webfaces.xml.
  - b) actionform.xml.
  - c) faces-config.xml.
  - d) webcontext.xml.

e) serverconfig.xml.

4. (CESPE - 2010 - TRE-BA - Analista Judiciário - Análise de Sistemas) Entre os itens que o padrão Java Server Faces (JSF) utiliza, estão os componentes, os eventos e a navegabilidade.

5. (FCC – 2013 – TRT/9 – Analista de Sistemas) Uma aplicação utilizando o framework JSF e a IDE NetBeans gera automaticamente dois componentes essenciais assim descritos:

I. É responsável por receber requisições dos componentes View do MVC, redirecioná-las para os beans gerenciados (managed beans) do componente Model do MVC e responder a essas requisições.

II. É o arquivo principal de configuração de uma aplicação web que utiliza o framework JSF. É responsável por descrever os elementos e sub-elementos que compõem o projeto, tais como as regras de navegação, beans gerenciados, configurações de localização etc.

As descrições I e II referem-se, respectivamente, aos componentes:

a) servlet Controller.java e ao arquivo faces\_config.xml

b) FaceletServlet e ao arquivo web\_config.xml.

c) FacesServlet e ao arquivo faces-config.xml.

d) servlet Controller e ao arquivo web-config.xml.

e) servlet Facelet e ao arquivo web.xml.

6. (CESPE - 2012 - ANAC - Analista Administrativo - Área 4) A validação de dados de um componente pode ser uma das funções de um backing bean, em uma aplicação JSF.

7. (FCC – 2012 – TST – Analista de Sistemas) O framework JavaServer Faces (JSF) é utilizado no desenvolvimento de aplicações web que utiliza o design pattern MVC. O JSF:

a) disponibiliza controles pré-construídos e código para manipular eventos, estimulando o uso de código Java convencional no componente View do MVC.

b) recebe requisições dos componentes da View do MVC, através do servlet FaceServerServlet.

c) armazena os mapeamentos das ações e regras de navegação em projetos JSF nos arquivos WEB-INF.xml e FACES-CONFIG.xml.

d) possui bibliotecas que suportam Ajax (Asynchronous JavaScript And XML).

- e) provê um conjunto de tags limitado para criar somente páginas HTML/XHTML.
8. (CESPE - 2013 - SERPRO - Analista - Desenvolvimento de Sistemas) O JSF provê uma linguagem de expressão exclusiva para acesso a objetos armazenados em bancos de dados.
9. (FCC – 2012 – TST – Analista de Sistemas) Para criar as páginas XHTML de uma aplicação JSF é possível utilizar um conjunto de bibliotecas de tags JSF. Algumas dessas bibliotecas são HTML, Core e Facelets. Considere os fragmentos de códigos abaixo, que utilizam tags dessas bibliotecas:

Fragmento de código I:

```
<h1>  
<ui:insert name="titulo">Título </ui:insert>  
</h1>
```

Fragmento de código II:

```
<h:inputText id="usuário" value="#{usuarioBean.usuario.  
nome}"/>
```

Fragmento de código III:

```
<h:selectOneMenu id="lista">  
  <f:selectItems  
    value="#{optionBean.optionList}"></f:selectItem>  
</h:selectOneMenu>
```

A correlação correta entre o fragmento de código e a biblioteca de tags utilizada é:

- a) I-Facelets, II-HTML e III-Core.
- b) I-Core, II-Facelets, e III-HTML.
- c) I-HTML, II-Core, e III-Facelets.
- d) I-HTML, II-Facelets, e III-HTML.
- e) I-HTML, II-HTML, e III-Core.
10. (CESPE - 2010 – TCU – Analista de Sistemas) No desenvolvimento de conteúdos para apresentação, o uso de facelets traz vantagens em relação ao uso de JSP. Uma delas é a maior modularidade, com o uso de templates e componentes compostos (composite).

11.(FCC - 2012 - TJ-PE - Analista Judiciário - Análise de Sistemas) No JSF, o componente Controller do MVC é composto por uma classe servlet, por arquivos de configuração e por um conjunto de manipuladores de ações e observadores de eventos. Essa servlet é chamada de:

- a) ControllerServlet.
- b) Facelet.
- c) HttpServlet.
- d) FacesConfig.
- e) FacesServlet.

12.(CESPE - 2012 – TJ/AL – Analista de Sistemas – B) Em um aplicativo Facelets, a tag f:ajax adiciona funcionalidades Ajax que necessitam de adicionais de codificação e configuração para as componentes de interface do usuário.

13.(FCC - 2012 - TRE-CE - Analista Judiciário - Análise de Sistemas) No ciclo de vida do Java Server Faces trata-se da fase na qual o componente deve primeiro ser criado ou recuperado a partir do FacesContext, seguido por seus valores, que são geralmente recuperados dos parâmetros de request e, eventualmente, dos cabeçalhos ou cookies gerados. Trata-se da fase:

- a) Restore View.
- b) Apply Request Values.
- c) Process Validation.
- d) Update Model Values.
- e) Invoke Application.

14.(CESPE - 2012 – TJ/RO – Analista de Sistemas – E) JNDI, parte do projeto de JSF, utiliza XHTML como tecnologia de apresentação dos dados, possibilitando a separação entre as camadas de negócio e de controle.

15.(FCC - 2012 - TRT - 11ª Região (AM) - Técnico Judiciário - Tecnologia da Informação) Sobre o framework JavaServer Faces é correto afirmar:

- a) A grande limitação do JSF é a dificuldade de integração com outros frameworks como Spring, JPA e EJB.

- b) Expression Language (EL) é a linguagem utilizada para apresentação de conteúdo em aplicações que utilizam JSF. Sua principal limitação é a impossibilidade de acessar valores e métodos em beans gerenciados.
- c) Facelets é uma parte da especificação JSF e também a tecnologia para implementar as regras de negócio em aplicações que utilizam JSF.
- d) Disponibiliza as bibliotecas de tags core e html para criar as páginas que compõem a interface do usuário.
- e) Define uma única forma para realizar a validação de dados em formulários JSP, por meio da implementação de uma classe de validação que estende a interface Validator.

16.(CESPE - 2013 – CPRM – Analista de Sistemas) Facelets são utilizadas para desenvolver visões (views) JavaServer Faces (JSF) com linguagem HTML e XHTML, em conformidade com a transitional document type definition, sendo, ainda, compatível com a biblioteca de tag JSF.

17.(FCC - 2011 - TRE-AP - Técnico Judiciário - Programação de Sistemas) O JSF extrai todos os valores digitados pelo usuário e guarda esse valor nos seus respectivos componentes. Se o valor digitado não coincidir com o componente, um erro vai ser adicionado na classe FacesContext e será mostrado na fase Render Response Phase.

No ciclo de vida do JSF trata-se de um evento típico da fase:

- a) Process Validations Phase.
- b) Restore View Phase.
- c) Apply Request Values Phase.
- d) Update Model Values Phase.
- e) Invoke Application Phase.

18.(CESPE - 2013 – INPI - Analista Judiciário - Análise de Sistemas) Quando registrado em JSF 2 (Java Server Faces), um managed bean permanece no escopo de session.

19.(CESPE - 2010 – TCU – Analista de Sistemas) Para suportar a construção de aplicações com Ajax e JSF, recomenda-se aos desenvolvedores de páginas que usem a tag <f:ajax>, relacionada ao processamento de pedidos http assíncronos.

20.(FCC - 2011 - TRE-RN - Técnico Judiciário - Programação de Sistemas) No ciclo de vida do JSF copiar os parâmetros de requisição para valores submetidos pelos componentes, é a tarefa típica da fase:

- a) Restaurar Visão (Restore view).
- b) Invocar aplicação (Invoke application).
- c) Aplicar valores de requisição (Apply request values).
- d) Processar validações (Process validation).
- e) Atualizar valores do modelo (Update model values).

21.(CESPE - 2010 – MPU – Analista de Sistemas) Uma aplicação web deve prover mecanismos de validação de dados. O JSF fornece vários validadores de dados padrões que podem ser utilizados no lado do cliente (client-side).

22.(FCC - 2010 - TRT - 22ª Região (PI) - Técnico Judiciário - Tecnologia da Informação) É um framework MVC utilizado no desenvolvimento de aplicações para a internet de forma visual, que utiliza o recurso de arrastar e soltar os componentes na tela para definir suas propriedades:

- a) Enterprise JavaBeans.
- b) JavaServer Faces.
- c) Java 2 Enterprise Edition.
- d) Servlets.
- e) Java Server Pages.

23.(CESPE - 2015 – TCU - Analista de Sistemas) A partir da interpretação do trecho JSF (JavaServer Faces), versão 2, no código a seguir, verifica-se que uma providência válida é configurar o managed-bean clientePage no arquivo faces-config.xml.

```
<f:view>
<h:form id="clienteForm">
<h:outputLabel for="informeNome" value="Informe Nome"/>
<h:inputText id="informeNome" value="#{clientePage.nome}"/>
<h:commandButton value="Nome do Cliente"
action="#{clientePage.cliente}"/>
</h:form>
</f:view>
```

24.(CESPE - 2014 - TJ-SE - Analista Judiciário - Análise de Sistemas) Antes de uma aplicação web desenvolvida nos moldes da JSF executar sua primeira página web, uma instância FacesServlet é executada, a fim de gerenciar as requisições dessa aplicação.

25.(FGV - 2013 – ALEMA - Analista de Sistemas) Com relação à especificação Java Server Faces (JSF), assinale V para a afirmativa verdadeira e F para a falsa.

- ( ) Visa substituir a especificação Java Server Pages.
- ( ) Java Server Faces são usadas como uma fachada para Servlets e Java Server Pages.
- ( ) Define um framework MVC (Model View Controller) para aplicações Web.

As afirmativas são, respectivamente,

- a) F, F e V.

- b) F, V e V.
- c) V, F e F.
- d) V, V e F.
- e) F, V e F.

26. (CESPE - 2014 - TJ-SE - Analista Judiciário - Análise de Sistemas) Em aplicações web nos padrões da JSF, é possível utilizar recursos Ajax para criar páginas dinâmicas, como, por exemplo, por meio da tag `f:ajax`, conforme apresentado na sintaxe abaixo.

```
<h:inputText value="#{bean.message}">  
  <f:ajax />  
</h:inputText>
```

27. (CESPE - 2014 - TJ-SE - Analista Judiciário - Análise de Sistemas) É possível utilizar XHTML no desenvolvimento de facelets para criar páginas web compatíveis com a JSF (JavaServer Faces) para apresentação dos dados. Na versão Java EE 7, essa forma de apresentação é mais indicada que a JSP (JavaServer Pages), uma vez que esta não suporta todos os novos recursos da versão Java EE 7.

## GABARITO

GABARITO



- |      |       |       |       |
|------|-------|-------|-------|
| 1. D | 8. E  | 15. D | 22. B |
| 2. C | 9. A  | 16. C | 23. C |
| 3. C | 10. C | 17. C | 24. C |
| 4. C | 11. E | 18. E | 25. A |
| 5. C | 12. E | 19. C | 26. C |
| 6. C | 13. B | 20. C | 27. C |
| 7. D | 14. E | 21. E |       |

## PRIMEFACES



Atualmente, com as recentes evoluções da plataforma Java EE, o JSF a partir de sua versão 2.0 incorporou várias soluções e funcionalidades existentes em projetos como o JBoss Seam, tornando se uma opção viável e competitiva para aplicações web onde se espera uma grande interatividade com o usuário aliada a um desenvolvimento e manutenção simplificados.

Com a extensa gama de bibliotecas de componentes ricos como PrimeFaces e RichFaces é cada vez mais fácil encontrar componentes prontos para solucionar seu problema. Nosso foco aqui é tratar do **PrimeFaces** muito utilizada atualmente para **aplicações web que utilizam JSF!** Professor, o que tem de mais nessa biblioteca?

**PrimeFaces** é uma **biblioteca de componentes JSF** contidas num arquivo jar, que não precisa de configuração e não requer qualquer dependência externa, está bom pra vocês??? :D

Desse modo, para se utilizar o Primefaces numa aplicação basta ter o jar nas dependências (classpath) e começar a usar seus componentes. Tudo que precisa ser feito após essa inclusão é importar o namespace na página que terá os componentes para usá-los.

A biblioteca PrimeFaces permite a chamada renderização parcial da página (Partial Page Rendering - PPR), parte da especificação do JavaServer Faces 2. Essa característica permite a escolha do que deve ser processado no ciclo de vida do JSF e o que deve ser renderizado ao usuário utilizando AJAX. Do lado do servidor, o Framework AJAX do Primefaces é baseado na especificação JSF, mas no lado do cliente, ao invés de usar o padrão como Mojarra ou o MyFaces, a biblioteca utiliza scripts baseados em JQuery. Muitos consideram isso uma grande vantagem, pois o JQuery é muito mais utilizado e suportado.

Como exemplo de utilização, podemos criar uma página simples com um botão que atualiza uma propriedade do tipo string na tela.:

```
<p:commandButton update="display" action="#{basicPPRBean.updateValue}" value="Update" />
<h:outputText id="display" value="#{basicPPRBean.value}"/>
```

O código acima apresenta a criação do botão (p:commandButton) atualizando a propriedade "display" com o método updateValue do objeto basicPPRBean. Aqui, temos que foram configurados os namespaces conforme código abaixo

```
xmlns:h="http://java.sun.com/jsf/html"  
xmlns:p="http://primefaces.org/ui"
```

PrimeFaces integra a API do jQuery Selector (<http://api.jquery.com/category/selectors>) com o modelo de referências a componentes do JavaServer Faces. A atualização e o processamento parcial das páginas (somente algumas partes atualizadas) dos componentes JSF podem ser feitas usando jQuery! Essa é uma baita vantagem do PrimeFaces! Essa possibilidade é trazida pela API chamada PrimeFaces Selector (PFS) API.

Outra coisa bem legal do PrimeFaces é que seus temas podem ser todos autocontidos como arquivos JAR. Muitos temas podem ser baixados de graça e ficam disponíveis para download no repositório do PrimeFaces (<http://repository.primefaces.org/org/primefaces/themes>). Aplicações podem até utilizar mais de um tema de uma vez, sem a necessidade de realizar uma requisição completa de página. Para isso a aplicação precisa conter o componente seletor de temas (theme switcher componente). O seletor padrão do PrimeFace permite mudança de tema automática, sem precisar de requisições de página, ou mesmo AJAX.

Outra característica interessante é o **inputMask**, que minimiza as chances de o usuário entrar com dados errados, bastando utilizar uma máscara que define como aquele campo deve ser preenchido. Um exemplo, para um campo de telefone, seira a máscara:

```
<p:inputMask value="#{inputMaskBean.phone}" mask="(99) 9999-9999"/>
```

Outro componente bem legal, galerinha, é o **dataTable**, que provê **tabelas ordenáveis**, com possibilidade de filtros, tudo com AJAX. Ainda, permite a edição dentro das células, sem necessidade de navegação entre páginas para atualizar os valores.

Finalmente, uma coisa que pode muito bem ser cobrada em prova, é a possibilidade de exportação de dados do dataTable. Isso é possível a partir da utilização do componente dataExporter, que permite a exportação nos formatos XLS, PDF, CSV, e XML.

É isso que temos para falar de Primefaces, galerinha!! Como é difícil ser cobrado (e não há questões que encontramos), temos que ser sucintos, pois não há tempo a perder!

# ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.